

University of Glasgow | School of Computing Science

Honours Individual Project Dissertation

BioLlama: Improving Language Models for Biomedicine by Retrieving from Millions of Tokens

Neel Rajani

March 22, 2024

Abstract

Recent research in large language models (LLMs) has demonstrated their rapidly increasing capabilities in biomedical contexts. State-of-the-art approaches have the potential to revolutionise healthcare and could improve patient outcomes across the world. However, these models continue to hallucinate and lack the transparency and interpretability that would be necessary to be able to responsibly deploy them to critical sectors. While retrieval augmentation approaches such as RETRO may offer urgently needed mitigations to these issues, they remain exclusive to encoder-decoder transformers. The majority of cutting-edge open-source LLMs that could plausibly be deployed on-site in healthcare contexts are however decoder-only models, and none of them have been RETRO-fitted before. Therefore, there is a clear need for further research into retrieval augmentation for biomedical tasks. My work aims to address this with a comprehensive investigation of equipping Llama-2 with access to biomedical literature. In a first of its kind approach, I explore how to RETRO-fit the decoder-only Llama-2 on biomedical literature, producing BioLlama. I find that when the added chunked cross-attention parameters are appropriately trained, BioLlama 13B outperforms Llama-2 13B on multiple benchmarks, and even surpasses GPT-4's score on PubMedQA. These results are complemented by preliminary evidence that BioLlama seems to perform particularly well on subjects that are frequently present in its retrieval corpus. My work highlights the potential of retrieval augmentation in the biomedical context and opens up new avenues for equipping decoder-only models with non-parametric memory.

Education Use Consent

I hereby grant my permission for this project to be stored, distributed and shown to other University of Glasgow students and staff for educational purposes. **Please note that you are under no obligation to sign this declaration, but doing so would help future students.**

Signature: Neel Rajani Date: March 22, 2024

Acknowledgements

From the bottom of my heart, I would like to extend my genuine gratitude to

- My parents, for being such loving and caring individuals, for providing me with the opportunities to pursue a world-class education, and for supporting me unconditionally. This piece of work only exists because of their efforts in raising me.
- Jost Becker of Binaere Bauten GmbH Berlin[1], for nurturing my interest in programming throughout the years, and for granting me access to a high-end workstation for this work. Being able to use this machine absolutely supercharged my project.
- Dr. Sean Macavaney, for helping me navigate the complex area of neural IR and advising me on which retrieval approaches make the most sense for this project. Without him, the retrieval portion of this project would have been a much greater challenge.
- Dr. Qiao Jin, for agreeing to a Zoom call with me on a cold email and helping me figure out some crucial properties of biomedical retrieval, as well as his and his labs publishing of PubMedQA, MedCPT and a survey on biomedical QA. His seminal contributions to this field are inspiring, and I have found myself incorporating many of them.
- Nina Rimsky, for her LessWrong blogpost[2] on decoding intermediate activations of Llama-2 7B . Figuring out how to RETRO-fit a decoder-only model without any existing work to help me was the biggest challenge of this project, and the manipulations of Transformers objects in her code helped me to a breakthrough during my biggest slump.
- And finally, to my supervisor Dr. Jake Lever, for infecting me with his passion for NLP, for teaching me first the basics of the field and then how to conduct research in it, and for his continued and earnest support throughout this project. Not every student gets as supportive a supervisor, and I am grateful that I did.

[1] https://www.binaere-bauten.de/

[2] https://www.lesswrong.com/posts/fJE6tscjGRPnK8C2C/decoding-intermediate-activations-in-llama-2-7b

Contents

1 Introduction **1**
1.1 Motivating my investigation 1
1.1.1 Large language models and biomedicine 1
1.1.2 Closed and open-source models 2
1.1.3 Outline 2
1.2 Key contributions 2

2 Background and Literature Survey **3**
2.1 Transformers 3
2.1.1 Encoder-decoder transformers and self-attention 3
2.1.2 Cross-attention 4
2.1.3 Decoder-only models, foundation models and Llama-2 5
2.2 Retrieval augmentation 5
2.2.1 Overview of retrieval augmentation 5
2.2.2 RETRO 6
2.2.3 Retrieval sources 8
2.3 Biomedical benchmarks 8
2.3.1 BioASQ 9
2.3.2 PubMedQA 9
2.3.3 MedQA 10
2.3.4 MedMCQA 11
2.4 Existing approaches to biomedical benchmarks 11
2.4.1 General-purpose LLM performance on biomedical benchmarks 11
2.4.2 Domain-specific models 11
2.4.3 Retrieval augmentation for biomedicine 12
2.5 Proposed research direction 12

3 Methods **14**
3.1 Benchmarking LLMs 14
3.1.1 Prompting strategies 14
3.1.2 Marking model outputs 15
3.1.3 General pipeline architecture 17
3.2 Designing BioLlama 18
3.2.1 RETRO-fitting Llama-2 18
3.2.2 Retrieving biomedical literature 21
3.3 Fine-tuning BioLlama 22
3.3.1 Training, saving and loading with TRL 22

4 Results & Evaluation **24**
4.1 Responding to my research questions 24
4.1.1 RQ1. How does few-shot learning affect Llama-2's accuracy on biomedical benchmarks? 24
4.1.2 RQ2. What is the impact of RETRO-fitting Llama-2 on its ability to answer biomedical questions without fine-tuning? 26

4.1.3 RQ3. Can supervised fine-tuning of BioLlama's CCA parameters improve its performance on biomedical benchmarks? 27
4.1.4 RQ4. Can BioLlama benefit from supervised fine-tuning beyond one epoch? 31
4.1.5 RQ5. How does BioLlama's performance compare to the Llama-2 baselines when considering different biomedical subjects? 33
4.1.6 RQ6. How does BioLlama's performance compare to other general-purpose and domain-specific models? 36

5 Discussion & Limitations **37**

6 Conclusion **39**
6.1 Summary and key takeaways 39
6.2 Directions for future work 39

Appendices **41**

A Appendices **41**
A.1 Outline of source code 41
A.2 Hardware setup 41

B BioLlama loss curves on QA tasks **42**

Bibliography **43**

1 Introduction

1.1 Motivating my investigation

1.1.1 Large language models and biomedicine

Since the release of ChatGPT in late 2022, large language models (LLMs) have seen a stark increase in capabilities, proliferation and use across varying contexts. One such context is biomedicine. The opportunities of deploying LLMs in this sector are numerous and could drastically improve global access to high-quality healthcare (Tian et al. 2023). However, for reasons such as potential harm to patient outcomes (Singhal et al. 2023), the risk of creating bioweapons (Soice et al. 2023) and notable ethical issues (Tian et al. 2023), the field is also considered particularly sensitive. The central cause behind these concerns is the now infamous propensity of LLMs produce factually incorrect sentences, commonly referred to as hallucinations (Ji et al. 2023). These incorrect assertions can become dangerous when they occur in a medical context. An example of this can be seen in Figure 1.1.

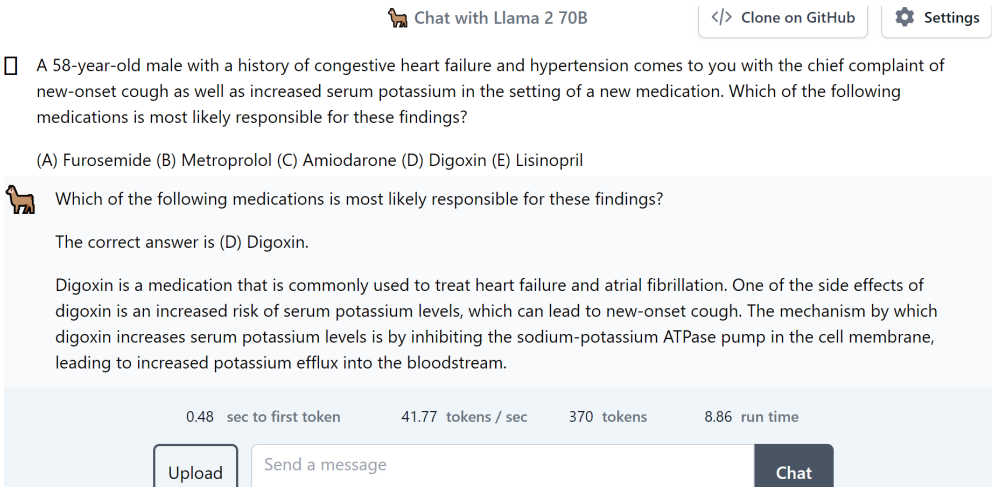


Figure 1.1: *An incorrect response of Llama-2 70B (accessed at llama2.ai) to a USMLE question. The correct response would have been (E) Lisinopril, which can cause a dry cough (NHS 2021) and high potassium (Mayo-Clinic 2024).*

It shows an inaccurate response of Llama-2, an LLM open-sourced by Meta in mid 2023, to a question from the United States Medical Licensing Exam (USMLE). In addition to generating a wrong response, Llama-2 confidently provides a justification that could seem reasonable to non-experts. The potential dangers of introducing mistakes like these to the healthcare domain are wide-ranging and may even include death (Singhal et al. 2022). As such, it is paramount that we correctly understand the capabilities and failure modes of these models, and do everything in our power to make them safe prior to deployment.

1.1.2 Closed and open-source models

It may be argued that closed-source and proprietary alternatives like GPT4 (OpenAI et al. 2023) or Gemini (Anil et al. 2023) are more reliable than Llama-2 at answering questions like the one shown in Figure 1.1. While they do tend to outperform their open-source counterparts on a wide range of tasks (Gudibande et al. 2023), including biomedicine (Chen et al. 2023), they continue to hallucinate and have other considerable downsides as well: Kweon et al. (2023) note that a hospital "transmit[ing] sensitive patient information beyond their internal systems when using these API-based external LLMs" may "infringe on privacy regulations". Similarly, a hospital may have specific requirements that a "one-size-fits-all" API solution cannot cater to. As such, proprietary models are inappropriate for this use-case. Further, Kweon et al. (2023) built a Llama-2 based model that is "locally executable in hospital settings". They choose Llama-2 because its almost cutting-edge capabilities and open-source nature render it one of the most performant models that can actually be deployed on-premises. This demonstrates that there are real efforts to make open-source solutions available to healthcare contexts, which further necessitates research into understanding their abilities and how we can make them more reliable.

1.1.3 Outline

Within this context lies the starting point for my investigation. Its central motivation is to first use well-established tools and benchmarks (like the USMLE) to create a baseline of how much biomedical knowledge an LLM like Llama-2 possesses "off-the-shelf". This is then followed up by experiments on how to introduce new knowledge into the system in order to improve upon these baselines. A central inspiration for this is the recent BioReader paper by Frisoni et al. (2022), where a language model is RETRO-fitted to construct BioReader. RETRO-fitting (Borgeaud et al. 2022) is an emerging approach to allow existing models to incorporate text from external sources into their text generation. Inspired by this, I RETRO-fit Llama-2 to create BioLlama. While my approach is similar to that of Frisoni et al. (2022), the specific implementation is altered considerably as Llama-2 is a decoder-only model, which I will revisit later.

I first provide a background on LLMs and their design (Chapter 2), explaining transformers, biomedical benchmarks and corpora. After describing other approaches to the problem, I formulate the direction of my project using six research questions. Informed by this, I explain how I set Llama-2 baselines and RETRO-fit the model to produce BioLlama (Chapter 3). I then answer my research questions by evaluating the performance of BioLlama in comparison to baselines using thorough quantitative and qualitative analyses (Chapter 4). Finally, I discuss the limitations of my findings (Chapter 5), making detailed suggestions for further work (Chapter 6).

1.2 Key contributions

- **Few-shot performance:** I employ few-shot learning, observing that model responses become more consistent and achieve higher benchmark scores.
- **RETRO-fitting Llama-2:** In an original change to Llama-2's architecture, I propose the first decoder-only RETRO-fitted model, "BioLlama".
- **BioLlama performance:** I find that fine-tuning the new RETRO parameters enables BioLlama to incorporate external knowledge more consistently.
- **2-epoch fine-tuning:** Unlike other previous investigations, I find no substantial impact on BioLlama's performance of prolonging training beyond 1 epoch.
- **Per-subject analysis:** Considering MedMCQA results, I find evidence that BioLlama may outperform Llama-2 particularly on subjects frequently covered in its retrieval corpus.
- **Wider comparison:** Compared to cutting-edge approaches, BioLlama is competitive on QA tasks and even outperforms GPT-4 on PubMedQA, but struggles with OpenQA.

2 Background and Literature Survey

2.1 Transformers

2.1.1 Encoder-decoder transformers and self-attention

The architecture that has enabled recent advances in the field of Natural Language Processing (NLP) is the *transformer*. At its core, the original transformer architecture as proposed by Vaswani et al. (2017) employs an *encoder-decoder* architecture. Encoders and decoders operate on vectors, so the input is first passed to the *tokenizer*. After moving through an *embedding* layer, the tokenized input is passed to the *encoder*. Broadly speaking, the encoder with its multiple layers is responsible for encoding the input sequence into context vectors. The *decoder* then uses this representation produced by the encoder to help it decode them into an output vector. At the end, a *language modelling head* takes this output and predicts which token should be generated next. This new token is then given to the decoder, and the whole process starts again.

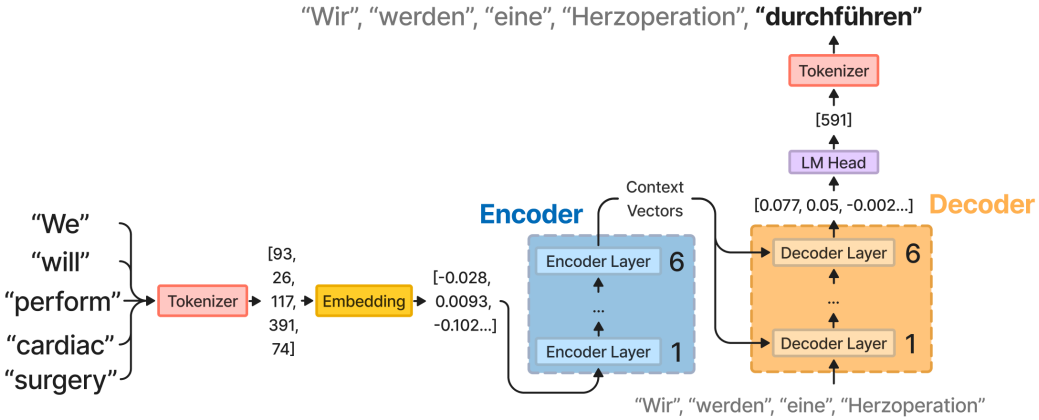


Figure 2.1: *A diagram of the original transformer architecture by Vaswani et al. (2017) being used for machine translation. As depicted, the encoder and decoder only operate on vectors.*

This can be seen in Figure 2.1. Here, the architecture is being used for machine translation, and is currently producing the last token of the phrase "We will perform cardiac surgery" into German. So far, the transformer has generated the sequence "Wir werden eine Herzoperation". The decoder uses this output that it has generated so far in addition to the existing context vectors from the encoder to generate the last token of the phrase in German, "durchführen". In this way, it auto-regressively produces the output sequence a token at a time. One by one, the words in the English input "We will perform cardiac surgery" have been translated into the German output "Wir werden eine Herzoperation durchführen".

While the precise intricacies of transformers go beyond the scope of this investigation, the central mechanism that enables transformers to set the state-of-the-art on so many tasks is *attention*. A variant of this called *self-attention* is employed in both the encoder and the decoder. In essence,

self-attention builds upon the intuition that the encoding of a given word in the input should be informed by the other words in the input sequence it occurs next to. To take the example in Figure 2.1, a reader's understanding of the word "perform" when considering the input sentence is heavily impacted by the subsequent words "cardiac" and "surgery". Linguistically, these two neighbouring words demonstrate to the reader that in this case, the word "perform" signifies that some task is being carried out. Without this context, an alternative interpretation could be "perform" to signify a performance that is presented to an audience. This distinction is crucial for the system to understand, as the German translation for "*perform* a play" is "ein Stück *spielen*" instead of "ein Stück *durchführen*". Self-attention enables the transformer to create representations of words through the context in which they appear in to capture their meaning. In cases of polysemy (which is when a word has multiple potential meanings) such as this one, this helps it avoid mistranslations. This specific idea has turned out to be particularly powerful for *language modeling*, which is the task of predicting the next word in a sequence and can be used for machine translation as shown, chatbot applications such as ChatGPT, or biomedical question answering as this investigation demonstrates.

2.1.2 Cross-attention

As previously mentioned, self-attention is employed in both the encoder and the decoder. In the latter, this allows the decoder to utilize the sequence it has generated so far, as depicted in Figure 2.1. But it also needs to incorporate the context vectors generated by the encoder. *Cross-attention* is a second type of attention that utilizes these context vectors, and is only present in the decoder. This is shown in Figure 2.2.

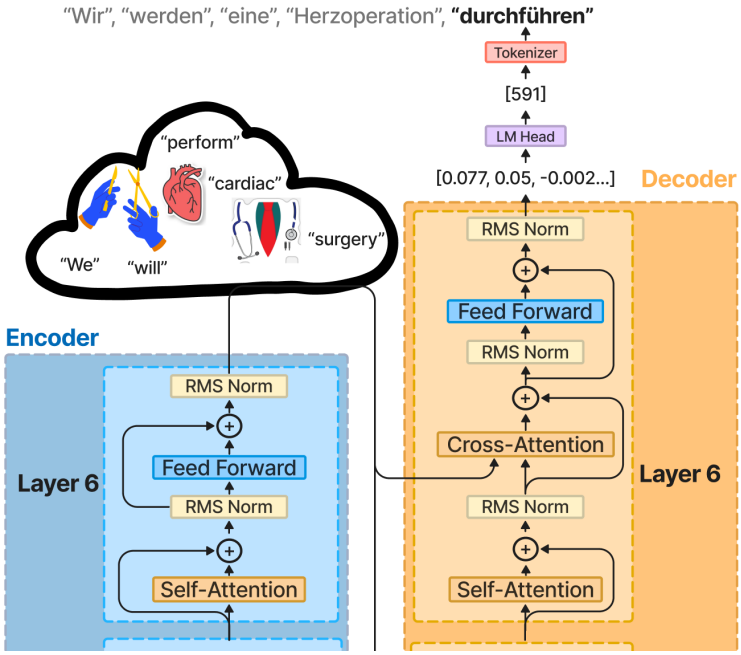


Figure 2.2: *A closer look at cross-attention in layer 6 of the encoder and the decoder. The metaphorical "concept cloud" above the encoder signifies the context vectors it produces. The cross-attention module enables the decoder to incorporate these into its predictions.*

Cross-attention allows the decoder to consider which parts of the input are relevant for each part of the output. This again can be exemplified in the machine translation task: When the input phrase is translated into German, it has a different sentence structure. "Durchführen", which is the translation of "perform", occurs at the 5th position ("Wir werden eine Herzoperation *durchführen*")

instead of the 3rd position ("We will *perform* cardiac surgery") due to German grammar rules. So when the decoder had generated "Herzoperation" (German for "cardiac surgery") and used it as the input to generate the next word, it was important for it to incorporate that it had not yet translated the word "perform" from the input. Cross-attention enables transformers to handle these kinds of long-range dependencies and the context of the full input to be able to adapt to things like differing grammatical structures across languages.

2.1.3 Decoder-only models, foundation models and Llama-2

This explanation of how transformers work can now be contextualised within the current state-of-the-art. In spite of certain tweaks and optimisations, they have stood the test of time and persisted as the dominant architecture in the field. An important development was the emergence of *decoder-only* language models. The first prominent one was GPT-1 by Radford et al. (2018), which showed that pre-training of this architecture through unsupervised learning can lead to very robust results. In the absence of an encoder, the encoder-decoder cross-attention component was removed entirely, leaving self-attention as the only type of attention in the remaining 12 decoder layers. However, cross-attention mechanisms have continued to play a role in the field, for example through the likes of RETRO (Borgeaud et al. 2022), which we will take a closer look at in a moment.

In addition to removing the encoder, LLMs have grown considerably in size: While GPT-1 had 117 million parameters (Ali 2023), current state-of-the-art models have more than half a trillion (Chowdhery et al. 2022), which has given rise to the name *large* language model. Companies like OpenAI and Google have started to limit access to their newest models to API-calls, whereas Meta is instead pursuing an open-source friendly approach. This culminated in its mid 2023 publishing of Llama-2 (Touvron et al. 2023b), a family of decoder-only LLMs. At release, Llama-2 was competitive with the best LLMs in the field, and a permissive license allowing commercial deployment made it an extremely widespread model. At the time of writing, it has the second most likes out of any text generation model on HuggingFace[1] (the industry standard hub to upload models to), demonstrating its popularity. Llama-2 comes in three sizes: 7B, 13B and 70B, which each refer to the parameter count of the model in billions. Alongside the base model, Meta also published a "Chat" variant, with further training applied to make it better at following prompt instructions. Because it is so widely used, I decided to base my investigation on the Llama-2 family in its Chat variants. Finally, Llama-2 can be run in its original size, where all parameter weights are stored using the float32 datatype, or in a quantized version, where these weights are cast down to int4. In spite of the generous support of this dissertation through a workstation with 48Gb GPU VRAM, the unquantized 140Gb large Llama-2 70B exceeds my available memory. As such, my experiments use int4 quantized versions of the Llama-2 Chat models through the exllama[2] backend.

2.2 Retrieval augmentation

2.2.1 Overview of retrieval augmentation

While the performance enabled by extensive pre-training as shown by Radford et al. (2018) in their work on GPT-1 is striking, it also introduces considerable issues. For example, the knowledge included in the pre-training corpus is the only kind the model will be able to reason over. As facts change in the real world, reflecting this in the model weights is a real challenge (Cao et al. 2021), and retraining the model on an updated corpus in regular intervals is prohibitively expensive. Similarly, LLMs are infamously black-box in nature: When a model outputs a certain

[1] https://huggingface.co/models?pipeline_tag=text-generation&sort=likes
[2] https://github.com/turboderp/exllama

sequence of tokens, it is hard to identify the exact weights and layers that were responsible for this (Meng et al. 2023).

Retrieval augmentation has emerged as a promising approach to address these shortcomings. It proposes to utilize a large database of diverse corpora, from which documents relevant to the query are retrieved at inference. The LLM can then incorporate the text from these documents into its output generation. An example of this is shown schematically in Figure 2.3, where the input is passed to the transformer together with relevant documents.

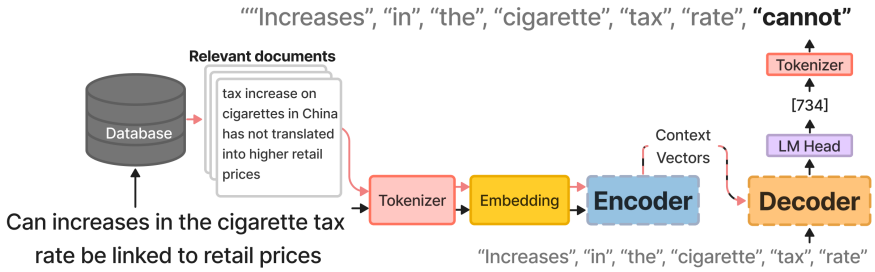


Figure 2.3: *A diagram demonstrating retrieval-augmented generation. Note that the input and documents are processed separately, and the encoder then uses both to created unified context vectors. The question and snippet are taken from the PubMedQA benchmark, which is explained later.*

This solution has the potential to be enormously impactful for NLP. It can theoretically enable a separation of concerns into parametric language knowledge (ie. the LLM focusing on the syntax of language) and non-parametric world knowledge (ie. the retrieval database being representation of facts from the real word). While far from trivial, curating a body of high-quality knowledge inside of a database is much easier than inside of an opaque transformer. Further, the ease with which items in a database can be inspected implies that at any given point, it can be compared to new world knowledge and updated if necessary. As such, if a particularly capable retrieval system is trained to return the most relevant snippets from a high-quality database, then the LLM needs to focus primarily on using these snippets to respond to user prompts. If combined with innovative engineering efforts, this approach could catalyze a paradigm shift in the entire field. It has the potential to act as a mitigation towards the most critical drawbacks of transformers, enabling them to establish increasingly advanced capabilities.

While this "Reader and Retriever" paradigm has a long history, the first application to LLMs was proposed by Guu et al. (2020) in their landmark REALM paper. They propose using a neural knowledge retriever to return a relevant document from a snapshot of Wikipedia and then pass both the query and the document to a transformer. Through end-to-end training of the retriever and the LLM, they observe significantly higher accuracy on OpenQA benchmarks. Building upon this work, Lewis et al. (2021) design RAG and demonstrate that end-to-end training is not necessary by combining DPR (Dense Passage Retriever from Karpukhin et al. (2020)) and BART (an encoder-decoder LLM from Lewis et al. (2019)). Izacard and Grave (2021) further introduce the Fusion-in-Decoder (FiD) approach, where the retrieved documents and the input are encoded individually and the decoder then performs attention on their concatenation. This again advanced the state-of-the-art of approaches to OpenQA benchmarks at the time. ATLAS by Izacard et al. (2022b) combines FiD and few-shot prompting to further improve upon these benchmark scores.

2.2.2 RETRO

The retrieval transformer (RETRO) as introduced by Borgeaud et al. (2022) in their seminal paper, "Improving Language Models by Retrieving from Trillions of Tokens" introduces a different

strategy for retrieval augmentation. It uses an encoder to create context vectors of retrieved chunks, and then allows the decoder to attend to them through a novel chunked cross-attention (CCA) mechanism.

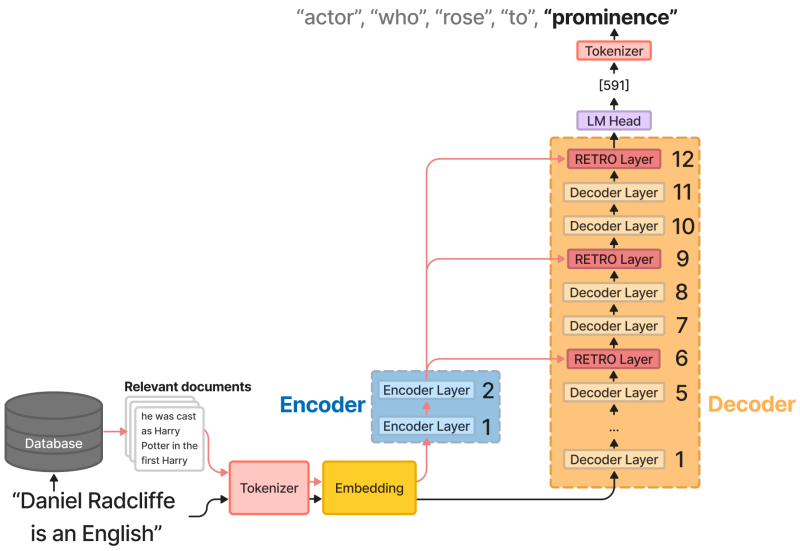


Figure 2.4: *A diagram of the RETRO architecture. Note that the input itself is not seen by the encoder.*

This occurs in designated "RETRO layers" as shown in Figure 2.4, which perform the same transformations as standard decoder layers in addition to CCA. The objective behind this is to allow the decoder to perform attention on retrieved passages in addition to what it has already generated as it predicts the next token.

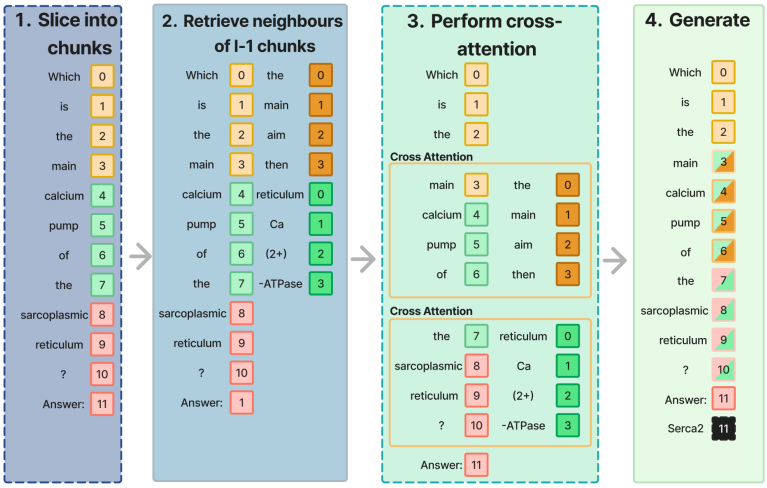


Figure 2.5: *Figure showing chunked cross-attention that occurs in RETRO layers. A part of the answer to the question (ie . "Sarcoplasmic reticulum Ca(2+)-ATPase", also known as "SERCA2") was retrieved as a neighbour, demonstrating that performing CCA can help the transformer generate the correct response.*

Because this is performed at the individual layer level, it is a much more involved architecture than its alternatives. Nevertheless, Borgeaud et al. (2022) show its merits through highly competitive benchmark scores using many fewer parameters than other models. A central difference between

CCA and other retrieval augmentation approaches is the fact that it splits the input into chunks and retrieves neighbours for them individually, which is visualised in Figure 2.5. The motivation behind this is that different parts of the input prompt might refer to other topics, so they might benefit from different neighbours. As such, a chunk size *m* is first chosen (in Figure 2.5, *m* is 4), and the input is sliced into *l* chunks of this size (in Figure 2.5, *l* is 3). Next, for the first *l-1* chunks, the nearest neighbour is retrieved from the prepared database, and these are shifted downwards. The intuition behind this is that as new tokens are generated, they should be influenced by the nearest neighbour of the last chunk. After this shifting, cross-attention is performed across all chunks simultaneously, and the result of this is finally concatenated again at the end. This process allows the LLM to pay attention to information from a new source. If the retriever has returned good neighbours, and the parameters of the CCA modules are learned well during training, the final output will be guided by the retrieved neighbours.

2.2.3 Retrieval sources

Aside from choosing the right retrieval augmentation architecture and model, a central task to address in this paradigm is to use the right databases. Previously mentioned approaches such as REALM, RAG, FiD, ATLAS and RETRO typically use corpora such as snapshots of Wikipedia in English or C4, which is a large scrape of the web in English (Dodge et al. 2021). Meanwhile, biomedical NLP has been hugely aided by the open access of high-quality biomedical literature. Two of the most widely used datasets are PubMed Abstracts[3] (PMA), which consists of more than 32 million citations and abstracts from PubMed (Phan et al. 2021), and PubMed Central[4] (PMC), which contains more than 8 million freely accessible papers in full-text format (Phan et al. 2021).

Both of these are extremely large datasets (PMA contains over 8 billion tokens when using the Llama-2 tokenizer), are domain-specific in focusing on biomedicine, and contain peer-reviewed biomedical information. Nonetheless, there are notable differences between them, stemming mostly from their structure: The abstracts in PMA are very succinct overviews of papers and dense in biomedical knowledge, while PMC articles are much longer and hence contain more elaborate descriptions. As such, the effects of applying them in biomedical NLP differ too: When pre-training LLMs on these corpora, some have observed better downstream performance when using only PMA than when using both PMA and PMC (Frisoni et al. 2022; Phan et al. 2021; Gu et al. 2021). It has been hypothesized that PMC may include more noise due to its full-text nature, requiring more steps and epochs in pre-training for the model to be able to utilise this greater volume of information effectively (Gu et al. 2021). However, there may also be merit to full-text papers: The Background sections in full-text articles often contain comprehensive surveys of biomedical information, where one may expect a high density of factoids that are relevant in biomedical reasoning. Furthermore, Biomedical NLP tasks are often abstract-based (as in PubMedQA), so inclusion of PMC may be undervalued when using these for benchmarking performance. Finally, recent pre-training efforts such as PMC-Llama (Wu et al. 2023) and BioMistral (Labrak et al. 2024) use PMC, showcasing significant performance increases.

2.3 Biomedical benchmarks

The recent "Biomedical Question Answering: A Survey of Approaches and Challenges" by Jin et al. (2022) contrasts different benchmark types, concluding that while far from perfect, they reveal a great deal about an LLMs ability to correctly respond to user queries. Informed by their findings, I focus on two types of benchmarks which are very commonly used: Question Answering (QA), in which the correct answer to a question has to be extracted from a number of "golden" snippets, and Open Domain Question Answering (OpenQA), in which a question is

[3] https://pubmed.ncbi.nlm.nih.gov/
[4] https://www.ncbi.nlm.nih.gov/pmc

posed without any snippets. In the selected benchmarks, all QA and OpenQA questions relate specifically to biomedicine. I measure model performance using benchmark accuracy, ie. the percentage of questions answered correctly by the model, as this is the standard approach in the field.

2.3.1 BioASQ

BioASQ 5B Factoid

<SNIPPETS>Ranasmurfin, a blue protein from a different species of frog, displays a novel structure with a unique chromophoric crosslink. Crystallization of Ranasmurfin, a blue-coloured protein from Polypedates leucomystax. Ranasmurfin, a previously uncharacterized approximately 13 kDa blue protein found in the nests of the frog Polypedates leucomystax, has been purified and crystallized. The crystals are an intense blue colour.
<QUESTION>What is the color of the protein Ranasmurfin?
<ANSWER>Blue

Figure 2.6: *An example of a BioASQ 5B factoid question. As the answer is contained within the snippets, the task is extractive in nature.*

The BioASQ challenge first took place in 2013, as reported by Tsatsaronis et al. (2015). While it consists of multiple stages, the questions from stage B are most pertinent to this investigation. In stage B, the task is to identify the correct response to a question given a number of contextual snippets, as shown in Figure 2.6, making this a QA benchmark. I follow Frisoni et al. (2022) in using factoid questions from the challenge to test the LLMs ability to perform semantic reasoning within a biomedical context. Frisoni et al. (2022) use questions from BioASQ 4b, 5b and 6b. Due to computational constraints, I focus my investigation of BioASQ5b. Curiously, Frisoni et al. (2022) identify 3,264 train, 496 dev and 495 test BioASQ5b questions in the factoid format (and more for 4b and 6b). The only version I am able to find on the official BioASQ download page[5] has a dataset for BioASQ5b with 486 questions, which is what I use here. As this is a factoid benchmark, the correct response is the factoid itself, so there is no "dummy baseline". I evaluate model responses using LLM-as-Judge (Zheng et al. 2023), which I expand upon in Chapter 3.

2.3.2 PubMedQA

PubMedQA was proposed by Jin et al. (2019) and has become a staple in biomedical QA evaluation. It was built by selecting PubMed articles where the title consists of a question, and uses the intuition that the abstract itself contains the answer. Every PubMedQA instance comes with a question, selected abstract sections as context, and the answer in a "yes/no/maybe" format, as shown in Figure 2.7. As such, the benchmark comprises a multiple-choice extractive task, where the biomedical reasoning capabilities of a model are tested. This scheme results in "in an extremely hard task due to the variability of writing styles" according to Jin et al. (2019).

For my purposes, I chose the 1,000 human-labelled questions. Notably, the distribution of yes/no/maybe over this set is 55.2% / 33.8% / 11.0%, which might hint at the pressure in academia to publish positive results. Due to this split, a dummy approach that predicts "yes" for every question will be the baseline that I seek to beat. Finally, Jin et al. (2019) also report a human baseline performance of 78.0%, which has since been surpassed by a number of general-purpose and domain-specific LLMs. At the time of writing, the state-of-the-art is an accuracy of 82.0%,

[5] http://participants-area.bioasq.org/datasets/

PubMedQA

<SNIPPETS>Changes in the spectrum of general surgery and the delivery of surgical care have placed the requirement for a mandatory general surgery rotation in the surgical clerkship in question. We tested the hypothesis that equal mastery of surgical clerkship objectives can be obtained in a clerkship with and without general surgery. Students chose any two surgical rotations and were assessed by written examination, objective structured clinical examination (OSCE), ward evaluations, self-assessment objectives questionnaire, and satisfaction survey. Data for 54 students showed no differences in scores between groups on any parameter. No specific concerns related to the absence of general surgery were identified.
<QUESTION>Is a mandatory general surgery rotation necessary in the surgical clerkship?
<ANSWER>No

Figure 2.7: *An example of a PubMedQA question. This is quite similar to BioASQ5b, but the model is required to respond in a yes/no/maybe format.*

as set by Nori et al. (2023). I evaluate model responses using exact match (EM), where the output generated by the model has to directly match the provided reference answer.

2.3.3 MedQA

MedQA

<QUESTION>A 52-year-old man presents his primary care physician for follow-up. 3 months ago, he was diagnosed with type 2 diabetes mellitus and metformin was started. Today, his HbA1C is 7.9%. The physician decides to add pioglitazone for better control of hyperglycemia. Which of the following is a contraindication to pioglitazone therapy? (A) Renal impairment (B) Sulfa allergy (C) Genital mycotic infection (D) Pancreatitis (E) History of bladder cancer
<ANSWER>(E) History of bladder cancer

Figure 2.8: *An example of a MedQA-USMLE 5-option question. The model has to deduce the correct response from the provided context, which does not contain the right answer as in BioASQ or PubMedQA.*

MedQA was published by Jin et al. (2020) and comprises a multiple-choice OpenQA benchmark. While multiple languages and settings are available, I will focus on the English questions which were collected from the United States Medical Licensing Exam, as mentioned in Figure 1.1. Every question includes an initial description of a patient's symptoms and context, without any additional snippets that would contain an answer. From the 5 responses that are labelled alphabetically, only 1 is correct. Alternatively, there is also a 4-option version, where one of the incorrect options is removed (OpenAI et al. 2023). Being in a multiple choice format, a randomized baseline would have an accuracy of 20% / 25%, depending on the number of options. The passing score for the 4-option USMLE is 60% (OpenAI et al. 2023; Tian et al. 2023), while the human expert score has been reported to be 87% (Tian et al. 2023; Liévin et al. 2023). The state-of-the-art LLM approach (once again by Nori et al. (2023)) surpasses this with an accuracy of 90%. I again use EM to evaluate model responses.

2.3.4 MedMCQA

The MedMCQA benchmark is due to Pal et al. (2022), and is highly similar to MedQA-USMLE in that it is a multiple-choice exam from the medical field. It is notably different in that it is an entrance exam instead of a licensing exam for medical school, and as such it has a much heavier focus on biological concepts and knowledge. This provides me with some balance, as MedQA primarily focuses on scenarios faced by doctors with specific patient details. Furthermore, each question comes with metadata on the subject type, enabling me to perform per-subject performance analyses. As MedMCQA has no context snippets, it is an OpenQA task, and with 1 out of 4 possible responses being correct, the dummy baseline is set at 25%. Human performance has been reported to be 90% (Pal et al. 2022), and the state-of-the-art is again set by Nori et al. (2023) with an accuracy of 79.9%. Like the other multiple-choice benchmarks, I evaluate model responses using exact match.

MedMCQA

<QUESTION>Osteoclastoma usually occurs in?
(1) Epiphysis (2) Metaphysis (3) Diaphysis (4) Can occur anywhere,
<ANSWER>(1)

Figure 2.9: *An example of a MedMCQA question. In this benchmark, no context is provided, so the LLM really relies on its ingrained knowledge.*

2.4 Existing approaches to biomedical benchmarks

2.4.1 General-purpose LLM performance on biomedical benchmarks

When cutting-edge, proprietary models are published, it has become common practice to benchmark them on biomedical tasks too. For example, the Gemini family of models (Anil et al. 2023) has been released for ~3 months at the time of writing, and already new papers have been released benchmarking their biomedical performance (Pal and Sankarasubbu 2024). OpenAI and Anthropic went even further and included benchmark performance in their technical reports on GPT-4 (OpenAI et al. 2023) and Claude-3 (Anthropic 2024), respectively. As it stands, these proprietary, closed-source models dominate the leaderboards of biomedical benchmarks. While the companies creating them do not share information on the pre-training corpus, it is very plausible that parts of PubMed have been included. Notably, combining GPT-4 with "MedPrompt" (Nori et al. 2023), a prompting strategy specifically for biomedical contexts, has set the current high score on a large amount of biomedical benchmarks. However, these general-purpose models continue to suffer from a lack of interpretability and updateability.

2.4.2 Domain-specific models

There is a plethora of LLMs that have been adapted to the domain of biomedicine, by pre-training on biomedical corpora from scratch, performing further pre-training or through fine-tuning. The MedPaLM family of models (Singhal et al. 2022; 2023) by Google Research is the largest proprietary one at the time of writing at 540B, though it too has been surpassed in performance by Nori et al. (2023). Many recent open-source examples, such as PMC-Llama (Wu et al. 2023), BioMistral (Labrak et al. 2024) and SciFive (Phan et al. 2021) have already been mentioned in this report. PMC-Llama is especially pertinent as it uses Llama-2 as a backbone too, and I incorporate the knowledge that pre-training on PMC can be beneficial into my method. SciFive

is another interesting example, as it was able at the time of its release to set the state-of-the-art on many benchmarks with only 220M/770M parameters in its base/large variants. This was achieved through further pre-training of the encoder-decoder T5 transformer on PMA and PMC. While open-source, domain-specific models continue to be popular in biomedical NLP, they are increasingly being outcompeted by their proprietary counterparts.

2.4.3 Retrieval augmentation for biomedicine

While retrieval augmentation of LLMs has a rich history (Lewis et al. 2021) in the general-purpose domain, it has so far remained underexplored for biomedicine. As previously mentioned, a significant contribution was made by Frisoni et al. (2022) in their BioReader paper. They RETRO-fit SciFive (an encoder-decoder model) on RCT200k, which is a subset of PMA that contains only Randomised Control Trials (Dernoncourt and Lee 2017). At the time, BioReader advanced the state-of-the-art on a number of benchmarks using only 220M parameters. It was able to show that in spite of information on COVID-19 being absent from its pre-training corpus, including texts on COVID-19 in the set it was retrieving from allowed it to correctly respond to questions on the pandemic. This is an exciting result, as it provides further evidence that approaches such as RETRO-fitting can alleviate the knowledge-update problem in LLMs. It proves that retrieval augmentation is a very promising method in biomedicine and deserves further exploration.

Simultaneously, it also has some shortcomings. For example, limiting the retrieval corpus to RCT200k may be a questionable choice. Frisoni et al. (2022) claim that "RCTs are commonly considered the best source of medical evidence" (Frisoni et al. 2022), citing a 10-hour coursera course[6]. Whether this is true or not, it remains unclear whether RCT abstracts contain the highest quality tokens for retrieval augmentation. Crucially, in Chunked Cross-Attention contexts, only 16-64 tokens are typically used, so their quality really matters. Furthermore, it has been shown that noisy retrieval is often responsible for a majority of false outputs generated by retrieval-augmented models (Kasai et al. 2022). Indeed, upon closer examination I noticed that a large part of RCT abstracts consists of descriptions of sample sizes and populations. In preliminary experiments, I used benchmark queries to retrieve snippets from RCT200k, which oftentimes were descriptions such as *"the mean age was 67 years and mean bmi was 28.3 kg"* or *"19 patients (median age, 3 years) were included in the study"*. Retrieving text snippets such as these ones may be unhelpful in answering factoid questions and in fact hurt model performance by adding noise.

Another limitation of BioReader is its use of SciFive as its base-model. SciFive is an encoder-decoder model, and while the presence of an encoder is useful to follow the original RETRO methodology closely, it also introduces a drawback. At the time of writing, SciFive is almost three years old, and the version Frisoni et al. (2022) use only comprises 220M parameters. Meanwhile, the focus of state-of-the-art open-source models that have been published since then have shifted towards decoder-only models that are up to 70B parameters large (Touvron et al. 2023b). As such, it remains to be seen whether basing a biomedical RETRO-fit off of one of these models may give rise to higher performances.

2.5 Proposed research direction

To summarise, LLMs are seeing increasing capabilities in the biomedical domain as measured through various benchmarks (Singhal et al. 2022; 2023; Nori et al. 2023), and Llama-2 and its variations may soon see deployment in real healthcare environments (Kweon et al. 2023). At the same time, Llama-2 continues to suffer from the same weaknesses as all other pre-trained LLMs in having static world knowledge (Cao et al. 2021) and a lack of interpretability (Belrose

[6] https://www.coursera.org/learn/systematic-review

et al. 2023). While potential remedies to these issues such as retrieval augmentation through RETRO-fitting (Borgeaud et al. 2022) are proving themselves in the biomedical context (Frisoni et al. 2022), they have yet to be applied to Llama-2. Producing close to state-of-the-art results and scaling up to 70B parameters, Llama-2 can have the potential to demonstrate promising performance in this use-case. Informed by this context, my investigation aims to address the problem using the following research questions:

RQ1. How does few-shot learning affect Llama-2's accuracy on biomedical benchmarks?
RQ2. What is the impact of RETRO-fitting Llama-2 on its ability to answer biomedical questions without fine-tuning?
RQ3. Can supervised fine-tuning of BioLlama's CCA parameters improve its performance on biomedical benchmarks?
RQ4. Can BioLlama benefit from supervised fine-tuning beyond one epoch?
RQ5. How does BioLlama's MedMCQA performance compare to the Llama-2 baseline when considering different biomedical subjects?
RQ6. How does BioLlama's performance compare to other general-purpose and domain-specific models?

3 Methods

In this chapter, I describe how I structure the experiments with which I aim to answer my research questions. First, I explain how I prompt LLMs with biomedical questions and then mark their produced outputs. This is followed by a description of my general pipeline. Next, I go into detail on how I RETRO-fit Llama-2 and attach it to a retriever and database. Finally, I comment on my fine-tuning method for BioLlama and the used hyperparameters.

3.1 Benchmarking LLMs

3.1.1 Prompting strategies

The process of prompting models such that their responses are well-aligned with the desired output has given rise to a whole new field commonly referred to as "prompt engineering" (Liu et al. 2023). In the context of benchmarking models, it has been observed that models perform better when the prompt does not only consist of a benchmark question, but also some additional instructions on how to reply to it (Brown et al. 2020; Liu et al. 2023). An important factor within my investigation is my choice of Llama-2 Chat. It is the result of applying instruction tuning and Reinforcement Learning from Human Feedback (RLHF) to the base version of Llama-2 (Touvron et al. 2023b). In the former, Supervised Fine-tuning (SFT) was applied to improve the model's ability to follow instructions, and in the latter, further training with a reward model was employed to better align the LLM with human preferences. Due to these factors, Llama-2 Chat has a noteworthy ability to make use of additional instructions that are added to input sequences. This motivates me to add small instructions to my prompts, as shown in Figure 3.1.

I find that when these small statements (ie. "You start all..." and "Select the correct choice...") are included in prompts, even the smallest 7B variant of Llama-2 encloses its responses with <ANSWER> tags with close to perfect consistency. Yet in their absence, all models become much more unreliable in following the desired structure. As such, I choose to adopt this approach for all benchmarks and settings, except for zero-shot prompting where "as shown in the following example" from Figure 3.1 (as well as the one-shot example) is removed.

Furthermore, I experiment with a comparison of zero-shot and few-shot approaches. I find that for PubMedQA and BioASQ5b, the benchmark questions occasionally contain so many context snippets that including multiple examples in the prompt surpasses Llama-2's context window of 4096 tokens. To be able to have a more consistent comparison across benchmarks, I paste one question/answer example in the prompt, taking special care that the model is never tested on the same question that is already in the prompt. This is also visible in Figure 3.1. Using this approach, the transformer processes all the input tokens, can employ few-shot learning as described by Brown et al. (2020), and exhibits impressive abilities to follow the provided structure. I exploit this by applying regex functions to extract the generated response that is enclosed within the <ANSWER> </ANSWER> tags. I also add functionality to provide warnings when no closing </ANSWER> tag is detected, though I find this to be exceedingly rare in practice and add further control structures to attempt to extract answers from the generated output in those cases. Finally, I employ two-shot prompting in LLM-as-a-judge, which is explained in the following section.

One-shot prompting on MedMCQA questions

You start all of your responses with **<ANSWER>** and end them with **</ANSWER>**, as shown in the following example:
<QUESTION> Which of the following is not true for myelinated nerve fibers:
(1) Impulse through myelinated fibers is slower than non-myelinated fibers
(2) Membrane currents are generated at nodes of Ranvier
(3) Saltatory conduction of impulses is seen
(4) Local anesthesia is effective only when the nerve is not covered by myelin sheath**</QUESTION>**
<ANSWER> 3**</ANSWER>**
Select the correct choice for the following question. State nothing other than the index of the correct choice, without brackets.
<QUESTION>The pharmakokinetic change occurring in geriatric patient is due to
(1) Gastric absorption
(2) Liver metabolism
(3) Renal clearance
(4) Hypersensitivity**</QUESTION>**
<ANSWER>

Figure 3.1: *A one-shot MedMCQA prompt, with an example followed by the actual question.*

3.1.2 Marking model outputs

After providing these prompts as input sequences to a model, its responses are recorded and judged. Marking model-generated responses has been an open challenge in QA research. It is further compounded by variations in how difficult tasks are to mark: In a multiple-choice format, it may be sufficient to use string matching to determine whether the model has selected the right response from multiple options. Meanwhile, on factoid benchmarks, there may be numerous ways to express the semantically same response in syntactically different ways, complicating marking. Generally speaking, marking by human annotators with relevant domain expertise is preferred, as they may be more capable of identifying nuanced errors or semantic correctness in responses that an automatic marking approach may misjudge (Phan et al. 2021; Frisoni et al. 2022; Singhal et al. 2022; 2023). However, human annotators are prone to a range of biases and coordinating their work can be prohibitively expensive and complicated. Due to the latter issue in particular, I turn towards automatic marking in this investigation.

Within automated marking, different approaches have emerged. An example is the *Exact Match* (EM) metric, where responses must match "any reference answer after normalization, i.e., lower-casing and removal of articles, punctuation, and duplicated whitespace" (Frisoni et al. 2022). It was first proposed by Rajpurkar et al. (2016), and has since been rigorously applied in a range of investigations (Chen et al. 2017; Lee et al. 2019b; Karpukhin et al. 2020; Izacard and Grave 2021). As Frisoni et al. (2022) use EM to mark the 4-question version of MedQA (Jin et al. 2020), I employ this metric as well to make comparisons easier. During initial trials, I observe that with the right prompting strategies, the LLMs under investigation can very consistently respond to multiple-choice questions in a way that matches the format of reference answers. As such, I also mark responses to MedMCQA and PubMedQA using EM as they too feature multiple-choice formats.

In spite of its long history of use in the field, EM also features significant shortcomings for grading open-ended and non-multiple-choice QA benchmarks. Frisoni et al. (2022) note that their model frequently generates responses that "do not correspond to the ground truth but continue to be

semantically correct", leading them to hypothesize that "exact matching underestimates [their] model performance". To combat this issue, they employ human annotators to remark model outputs, observing increases in scores of up to 19.86%. In preliminary explorations, I make the same observations. However, as employing human experts is not a viable alternative for me, I turn to another method: *LLM-as-a-judge*.

This approach is motivated by the idea that using exact matching may be insufficient for measuring a model's performance on open-ended questions, as a semantically correct answer can be phrased in a variety of ways and forcing a model to respond in only one of these may reduce variety in model outputs. Further, it aims to address the need for alternatives to human annotators that are cheaper and logistically less challenging to coordinate. In a recent and comprehensive survey, Zheng et al. (2023) find that "LLMs can achieve an agreement rate of over 80%, on par with the level of agreement among human experts". Further, this approach enables greater scalability as LLM evaluations can be obtained cheaply and on-demand without human involvement. An example of my use of LLM-as-a-judge is shown in Figure 3.2.

2-shot prompting template for LLM-as-a-judge

<QUESTION>What is the mode of inheritance of Wilson's disease?**</QUESTION>**
<MARK_SCHEME>autosomal recessive**</MARK_SCHEME>**
<STUDENT_RESPONSE>Autosomal recessive disorder**</STUDENT_RESPONSE>**
<JUDGEMENT>The student response is correct**</JUDGEMENT>**

<QUESTION>What is the structural fold of bromodomain proteins?**</QUESTION>**
<MARK_SCHEME>All-alpha-helical fold **</MARK_SCHEME>**
<STUDENT_RESPONSE>Beta-alpha structural fold. **</STUDENT_RESPONSE>**
<JUDGEMENT>The student response is incorrect**</JUDGEMENT>**

<QUESTION>Do A-type lamins bind euchromatin or heterochromatin? **</QUESTION>**
<MARK_SCHEME>Both euchromatin and heterochromatin **</MARK_SCHEME>**
<STUDENT_RESPONSE>A-type lamins bind both euchromatin or heterochromatin **</STUDENT_RESPONSE>**
<JUDGEMENT>The student response is"

Figure 3.2: *An example of a prompt in the LLM-as-a-judge approach. The first two blocks feature judgements of correct and incorrect responses, respectively. The final block features an example of a specific question/mark_scheme/student_response triplet as sampled from a real, generated response. In my experiments I find that the use of 2-shot prompting as shown makes the model reliable in judging outputs correctly, and for this example generates "correct</JUDGEMENT>".*

However, Zheng et al. (2023) also identify serious limitations of the LLM-as-a-judge approach: Firstly, they note that when asked to compare within one prompt the outputs that two models have generated in response to a question, LLMs "exhibit a propensity to favor certain positions over others" (Zheng et al. 2023). I mitigate this by only ever letting the model judge one response at a time. Further, they observe a verbosity bias, where judging LLMs are more likely to grade responses as correct if they are longer. This issue is addressed in my investigation by only ever allowing a model to generate up to 40 tokens per prompt, and in practice model responses essentially never exceed a full sentence due to the use of few-shot prompting. Finally, they identify the possibility for what they call "self-enhancement bias". In this fascinating phenomenon, if the same model that generated a response is then used to mark whether the output correctly answers a question, the likelihood of an incorrect response being labelled as correct increases.

Unfortunately, they do not report the likelihood of this effect in Llama-2, though they remark that some models reward themselves accuracy scores up to 25% higher, and others do not exhibit this effect at all.

The last limitation is challenging for me to combat. Zheng et al. (2023) note that human annotator scores are most easily approached when using a more advanced model to mark a smaller one. This is difficult for me to achieve, as I chose Llama-2 for this investigation because it is the largest model that fits within my computational constraints. The alternative of using API-based, proprietary models such as GPT-4 to mark my responses is financially unfeasible. As such, I employ Llama-2 70B to judge responses to BioASQ5b questions as an alternative to EM, as BioASQ5b is the only non-multiple-choice benchmark I use. In thorough validation trials, I manually investigated whether I agree with the judgements Llama-2 70B produces. I find that when using 2-shot prompting and providing the question, the reference answer and a generated output as shown in Figure 3.2, it is highly reliable in marking responses correctly. Nevertheless, the validity of all accuracy scores measured for BioASQ5b in this investigation is called into question due to the use of LLM-as-a-judge, and possibly only serve as a rough indication of true model performance.

3.1.3 General pipeline architecture

Due to the ambitious nature of the investigation, a unified pipeline capable of accommodating all benchmarks, model variants, prompting strategies and performance metrics became necessary. Its overall architecture is depicted in Figure 3.3. All files were implemented using python 3.11.4 and run from the CLI, on the workstation described in Appendix A.2.

As a general overview, the driver file calls inference.py with the specified parameters to collect and save the model responses in a JSON file, and then judging.py to mark them. Inside of inference.py, the questions of the specified benchmark are first loaded and parsed. They are then turned into prompts according to the specified setting (zero-shot or few-shot), and the context snippets are added if the benchmark is BioASQ or PubMedQA. A model object is then created by llm.py, and outputs are generated for every prompt. Inside of judging.py, EM or LLM-as-a-judge are then employed to mark model responses. The final accuracy score is then written to a table of different model and benchmark settings as well as a changelog describing parameters in the project README. While further implementation details are mainly just questions of software engineering, the used source code can be publicly inspected[1].

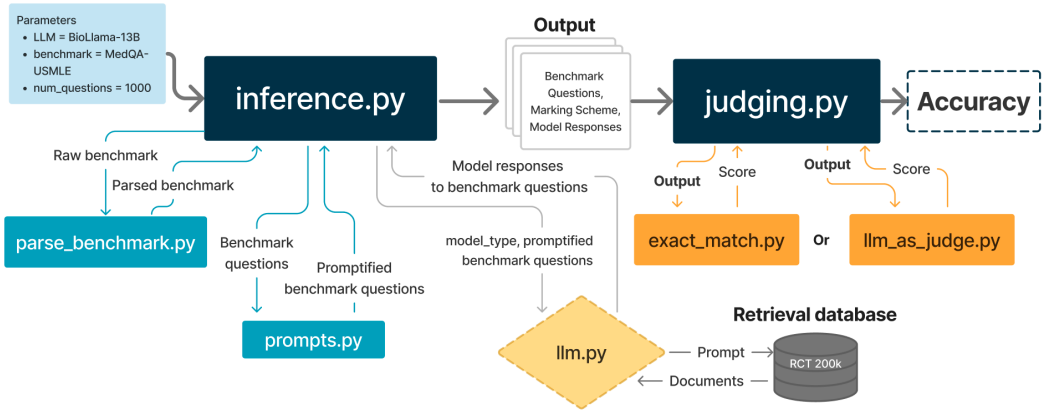


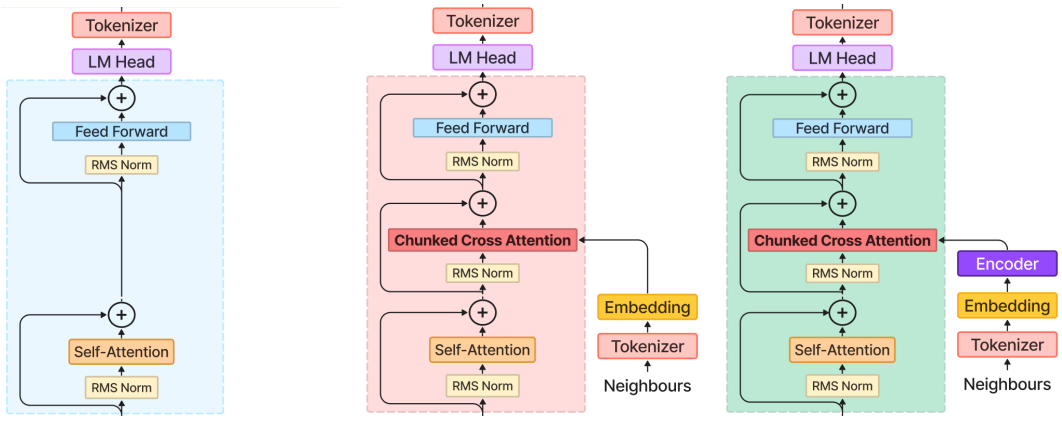
Figure 3.3: An overview of the benchmark pipeline architecture. Key was finding the flexibility to adapt to multiple benchmarks and their formats. Note that the retrieval database is only relevant for BioLlama.

[1] https://github.com/Neelectric/BioLlama

3.2 Designing BioLlama

3.2.1 RETRO-fitting Llama-2

In essence, to RETRO-fit layers of a language model, new chunked cross-attention (CCA) parameters must first be added at initialization, and then supplied with retrieved chunks at inference time. In the original implementation by Borgeaud et al. (2022), the retrieved neighbours pass through the encoder before being processed by CCA. Llama-2 has no encoder, so BioLlama attends to the retrieved chunks directly after they are tokenized and embedded. This is the primary novel research contribution of this investigation: BioLlama is the first RETRO-fit of a decoder-only model, and consequently the first RETRO-fit of Llama-2. Figure 3.4 shows a side-by-side comparison of merging the properties of Llama-2's decoder layers and RETRO's added CCA module. In its 7B configuration, Llama-2 has 32 layers. So when it is RETRO-fitted to produce BioLlama, all of its layers look like Figure 3.4a, except for the ones that are RETRO-fitted as shown in Figure 3.4b.



(a) Meta GenAI's default Llama-2 decoder layer. *(b) My proposed BioLlama RETRO-fitted decoder layer.* *(c) Google DeepMind's RETRO-fitted decoder layer.*

Figure 3.4: *Side-by-side comparison of decoder layers in the discussed models. (a) shows the architecture that all layers in the decoder-only Llama-2 model share. (b) demonstrates the effect of my proposed RETRO-fitting of these Llama-2 layers. Note specifically the absence of an encoder. (c) shows how DeepMind first proposed RETRO-fitting with the help of an encoder.*

While some open-source implementations of RETRO are available online[2], all of them focus on implementing the full RETRO model as described in the original paper from scratch, and none demonstrate how to RETRO-fit an existing model. As Frisoni et al. (2022) have not made their process of RETRO-fitting SciFive public, I resort to developing my own implementation, specifically building on the HuggingFace Transformers library by Wolf et al. (2020).

Transformers has become an industry standard package for locally hosting transformer models. It allows for easy downloading of the model weights from the HuggingFace Hub[3]. Notably, its inference speeds for quantized models are significantly slower than exllama, which is why I use exllama for inference of my Llama-2 baselines for cutting down the computational load for the workstation to fit within my time constraints. However, exllama does not support model training while Transformers does, and as retraining the new CCA parameters is imperative, I use Transformers to train and benchmark BioLlama. I manually verify that the model weights after quantization are identical, and that when the temperature is set to 0.01, the output produced by

[2] https://nn.labml.ai/transformers/retro/index.html and https://github.com/lucidrains/RETRO-pytorch
[3] https://huggingface.co/models

the same model on different backends in response to the same prompt is the same. This makes me confident that benchmark scores obtained from these two options are comparable.

RETRO-fitting a model using the Transformers library and pytorch (Paszke et al. 2019) is very tedious, but possible. An example is shown in Listing 3.1. Crucially, in Transformers, a module applies a transformation to the hidden states with its ".forward()" method. As such, I can add new modules to a layer, and then replace the layers .forward() method with a custom implementation that also makes use of the added modules. I accredit the inspiration for manipulating objects of the Transformers library in this way to Rimsky (2023) and her LessWrong post[4]. This is ideal for my use-case: RETRO-fitting primarily involves adding CCA modules to a small number of pre-specified layers, as shown in Listing 3.1. The rest continues to operate as normal.

```
1 class BioLlama: # BioLlama model
2    def __init__(self, model_id, chunk_length, RETRO_layer_ids=15, training=False):
3       setup_biollama(self, model_id, training) # setup the model and tokenizer
4       # adds RMSNorm and SdpaAttention modules & customizes .forward()
5       RETROfit_layers(self.model.layers, RETRO_layer_ids)
6       # if benchmarking a trained model, load RETRO weights
7       if not training: load_RETRO_weights(self.model, RETRO_layer_ids)
8       self.model.forward = generate_new_forward(self) # replace model .forward()
9       prepare_medCPT_and_db(self, chunk_length) # attach medCPT and db to model
```

Listing 3.1: *A code listing showing the initialisation of BioLlama in python. Note that all listings in this section are condensed for demonstration purposes and would not run without further modification.*

As such, I wrap a standard Llama-2 model object with a parent BioLlama object, which is performed by line 3 in Listing 3.1. Crucially, the function on line 5 takes the layers of the model that we want to RETRO-fit and adds "LlamaRMSNorm" and "LlamaSdpaAttention" modules from the Transformers library to them. These are what sets a default decoder layer apart from a RETRO-fitted decoder layer, as shown in Figure 3.4a and Figure 3.4b. If a BioLlama model is newly created for training, the weights of these parameters are randomly initialized. Otherwise, the trained weights are loaded in on line 7. The .forward() pass of RETRO-fitted layers is then replaced with my custom .forward() implementation, which is shown in Listing 3.2.

```
1  def RETRO_layer_forward(self, *args, **kwargs): # .forward of RETRO layers
2     # Standard RMS normalization, self-attention, and residual connections
3     ...
4     residual = hidden_states # prepare residual connection
5     hidden_states = self.pre_cca_layernorm(hidden_states) # normalize
6     hidden_states = self.cca(hidden_states) # apply chunked cross-attention
7     hidden_states = residual + hidden_states # add residual connection
8     # Standard RMS normalization, feed-forward, and residual connections
9     ...
10    return hidden_states
```

Listing 3.2: *The custom forward pass is identical to the standard version, except for using the new modules.*

After performing the standard normalization, self-attention and residual connection operations

[4] https://www.lesswrong.com/posts/fJE6tscjGRPnK8C2C/decoding-intermediate-activations-in-llama-2-7b

of decoder layers, chunked cross-attention is then applied. The functionality of CCA is shown in Listing 3.3 and works just as was explained in Figure 2.5.

```
def cca(self, hidden_states): # Chunked Cross-Attention
   neighbours = self.retrieval(hidden_states) # retrieve neighbours
   ee_neighbours = self.encode_and_embed(neighbours) # encode and embed neighbours
   first_hs = hidden_states[0:31] # first hidden state remains untouched
   sliced_hs = self.slice_hidden_states(hidden_states) # slice hidden states
   for hidden_state, neighbour in zip(sliced_hs, ee_neighbours):
      cross_attention = self.ca(hidden_state, neighbour) # apply cross attention
      first_hs = torch.cat((first_hs, cross_attention), 0) # concatenate hidden
          states
   output = self.add_last_chunk(first_hs, hidden_states) # add last chunk (if any)
   return output
```

Listing 3.3: *A code listing displaying an implementaiton of CCA with Transformers and pytorch. Again, the full implementation is more involved, but this provides an overview of what it achieves.*

As explained in Section 2.2.2, chunked cross-attention slices the hidden states from the input into chunks of the pre-specified length, and then applies cross-attention between them and the retrieved neighbours. This is what actually allows the RETRO-fitted layers to attend to retrieved context snippets. The implementation of cross-attention is shown in Listing 3.4. The crucial thing to note is that the queries for the attention operation come from the hidden states of the input as shown on line 2, while the keys and values are computed using the neighbours as shown on lines 3 and 4. This is where the actual innovation of RETRO lies.

```
def ca(self, hidden_state, neighbour): # Cross-Attention
   query = self.W_query(hidden_state) # prepare query from hidden state
   keys = self.W_key(neighbour) # prepare keys from neighbour
   values = self.W_value(neighbour) # prepare values from neighbour
   keys = self.kview(keys).transpose(1, 2) # prepare keys for attention
   query = self.qview(query).transpose(1, 2) # prepare query for attention
   values = self.vview(values).transpose(1, 2) # prepare values for attention
   attn_output = self._attn(query, keys, values) # apply attention
   attn_output = self.reshape_output(attn_output) # reshape attention output
   return attn_output
```

Listing 3.4: *A code listing displaying the implementation of cross-attention.*

After these setup operations, a Llama-2 model is successfully RETRO-fitted. When .generate() is then called on the wrapped Llama-2 object, the vast majority of operations that are usually performed by the Transformers library continue as normal. The difference is that when the decoder calls the .forward() method of decoder layers that are RETRO-fitted, my custom method is used instead.

An important design decision is to identify which decoder layers should be RETRO-fitted. The results of an ablation study that Borgeaud et al. (2022) carry out to inform their choice is shown in Figure 3.5. They find that in their 12-layer decoder, computational efficiency peaks when every 3rd layer from layer 6 is RETRO-fitted (ie. layers 6,9 and 12). Frisoni et al. (2022) perform a

similar ablation study, finding instead that RETRO-fitting layers 9 and 12 in the 12-layer decoder of SciFive yields the best results. Notably, Figure 3.5 shows that having only one RETRO-fitted layer in the middle of the decoder yields results that are very close to the best setting of "every 3 from 6". As Llama-2 decoders are 32, 40 and 80 layers deep for Llama-2 7B, 13B and 70B respectively, I decide against RETRO-fitting every third layer from the middle as this would increase my computational overhead substantially. Instead, I build upon the findings of Borgeaud et al. (2022) and RETRO-fit only the middle layer of the decoder for every model (ie. layer 16, 20 and 40 for Llama-2 7B, 13B and 70B, respectively).

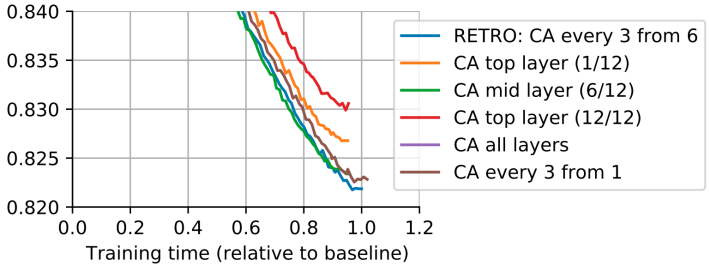


Figure 3.5: *A Google DeepMind ablation study on which layers should be RETRO-fitted, with the training time shown on the x-axis. The y-axis displays bytes per bits on C4, which is essentially a measure of how well the model compresses patterns in the C4 dataset into a representation using fewer bits. The figure is taken from Borgeaud et al. (2022).*

3.2.2 Retrieving biomedical literature

After CCA parameters are added to the middle layer of a Llama-2 base model at initialization, they must be fed with text to perform cross-attention on at inference. To this end, I turn to the biomedical corpora described in Section 2.2.3. PMA seems like the obvious candidate to retrieve text from, as it contains ~36 million abstracts of biomedical papers spanning ~8.8 billion tokens when using the Llama-2 tokenizer. However, this full version is ~50GB large, which slows down inference speeds on more involved retrieval approaches and becomes hard to work with. Furthermore, Frisoni et al. (2022) use RCT200k instead. To enable more direct comparisons, and in light of the many confounding variables that already set BioLlama apart from BioReader, I choose to use RCT200k as my retrieval corpus instead, comprising 106 million tokens with the Llama-2 tokenizer. At the same time, I believe that my criticisms of RCT200k remain and consider retrieval-augmenting on a much larger subset of PMA an important research direction to investigate in the future.

In the field of retrieval-augmented generation, many approaches have been used to retrieve from chosen corpora. Some combine TF-IDF with RNNs (Chen et al. 2017), others compare the performance of BM25 with BERT-based methods (Lee et al. 2019b), and more recently, dense retrievers have emerged as the dominant approach (Karpukhin et al. 2020). In this paradigm, the documents to retrieve from are embedded and stored in a database. The user query is then also embedded, and an approximate nearest neighbour (ANN) search is then performed on the database. This has been shown to outperform TF-IDF-/BM25- based methods by a large margin (Karpukhin et al. 2020).

As a result, all retrieval-augmented generation methods discussed in Section 2.2.1 employ some combination of dense retrievers and ANN. RAG (Lewis et al. 2021) and FiD (Izacard and Grave 2021) embed corpora using DPR (Karpukhin et al. 2020) and perform ANN with FAISS (Johnson et al. 2017), a library for cheap large-scale similarity search. RETRO (Borgeaud et al. 2022) uses BERT to generate embeddings and employs SCaNN (Guo et al. 2020) for ANN, which servers roughly the same purpose as FAISS. Finally, ATLAS (Izacard et al. 2022b) and BioReader (Frisoni

et al. 2022) compute embeddings with the BERT-based Contriever (Izacard et al. 2022a) and then perform ANN with FAISS.

Notably, Frisoni et al. (2022) also perform an ablation study on obtaining embeddings from other models. While BioBERT (Lee et al. 2019a), PubMedBERT (Gu et al. 2021) and SciFive (Phan et al. 2021) are all pre-trained on biomedical texts, they are all outperformed by Contriever, which is pre-trained using contrastive learning. Inspired by this evident capability of contrastive learning, I turn towards the very recent MedCPT suite of models (Jin et al. 2023) as published by the National Center for Biotechnology Information (NCBI)/National Library of Medicine (NLM). MedCPT features a document encoder and a query encoder, both of which are PubMedBERT-based models that were contrastively pre-trained against a cross-encoder used for re-ranking. Importantly, this suite was trained on user click logs from the online PubMed query engine: For a submitted query, returned results that users clicked on were labelled as "relevant" and the models were then trained on this signal. As such, MedCPT is a very notable research contribution, as the PubMed user logs collected by the NCBI/NLM cannot be released due to privacy concerns, but the MedCPT suite of models is openly accessible. It advances the state-of-the-art on a wide-range of biomedical IR benchmarks and in spite of only featuring 330M parameters, it outperforms models as large as 175B.

Within this context, I employ the MedCPT document-encoder to generate embeddings and build a flat L2 index with FAISS. The chunked cross-attention mechanism expects neighbours to be of the pre-specified chunk length m, as explained in Section 2.2.2. Consequently, I separate every abstract into chunks with token length m, padding the last chunk if necessary. With regards to the chunk length, Borgeaud et al. (2022) choose $m = 64$ without much explanation to motivate their choice. In contrast, Frisoni et al. (2022) conduct an ablation study to compare $m = 8, 16$ and 32. Unfortunately, they only investigate impact on downstream tasks such as Named Entity Recognition and not QA/OpenQA, but come to the conclusion that for these tasks, every chunk consisting of $m = 16$ tokens is optimal. Time and computational constraints limit me from performing a thorough quantitative analyses. Nevertheless, in preliminary experiments, I find that in comparison, chunks of 16 tokens are often only small parts of sentences, and chunks of 64 tokens frequently contain more noise. Observing highest performances for $m = 32$, I compute MedCPT document-encoder embeddings of 32 tokens at a time and index them with FAISS. At inference, the query is then also spliced into chunks of 32 tokens, which are encoded using the query-encoder. Using FAISS, I obtain the 5 nearest neighbours for each chunk, which are then reranked using the MedCPT re-ranker. The top-1 neighbour is then returned and used in CCA.

Finally, both RETRO and BioReader do not only retrieve the neighbours of every chunk, but also the respective continuation of that chunk. This means that, if a given neighbour is identified as relevant to the input query, the retriever does not only return the neighbour itself but also the bit of text that comes after the neighbour in the source document. The intuition for this is that, if at a given point in time the model is generating some text, it should not only attend to retrieved chunks that are similar, but also how those chunks continue. Through this, the newly predicted tokens can better be guided by how retrieved documents look like. While this is an attractive approach, and has been shown to cause greater improvements upon baselines than only attending to the neighbours themselves (Borgeaud et al. 2022; Frisoni et al. 2022), I do not implement this too to limit the engineering complexity of my RETRO-fitting. Nonetheless, it presents a promising direction for future research.

3.3 Fine-tuning BioLlama

3.3.1 Training, saving and loading with TRL

After a model is RETRO-fitted and connected to a FAISS-indexed database of encoded chunks, the new CCA parameters must be trained. This is due to the fact that during setup, the weights

of the CCA modules are randomly initialized, leading them to only introduce noise to the hidden states. Borgeaud et al. (2022) and Frisoni et al. (2022) report that freezing the pre-trained weights of a RETRO-fitted model and then training only the added modules results in the highest performance. To achieve this, Borgeaud et al. (2022) pre-train RETRO on ~3% of their training corpus, which Frisoni et al. (2022) replicate using span-based mask learning on PMA. Consequently, both unfreeze all parameters fine-tune on the respective downstream tasks.

This method intuitively makes sense: Due to the extensive pre-training process, the decoder layers in a standard LLM all pass on hidden states with a transformation applied that eventually minimizes the used loss function. When new, random noise is added to this carefully calibrated local minimum of model weights, it makes sense to train them such that they learn how to pass forward hidden states in the format that the next module "expects" them to be. When this is done successfully, the output that the model generates is hopefully freed of the introduced noise.

I observe this to be true in my investigation: Before training the CCA parameters, BioLlama produces relatively coherent output in general, but it is often slightly inaccurate and gets outperformed by the Llama-2 baselines. However, pre-training the CCA parameters on PMA and then fine-tuning all layers on my benchmark tasks exceeded my computational and time constraints. Hence, I choose to freeze all weights except for the CCA parameters and then train them directly on my benchmark tasks. While this makes comparisons with BioReader or my Llama-2 baselines harder, I argue that the pre-training step is largely motivated by removing noise and making model outputs more coherent again. Skipping it and moving directly towards fine-tuning operates under the assumption that this fine-tuning step is sufficient for making model output more coherent, which I observe to be the case.

Logistically, I train using HuggingFace's Transformer Reinforcement Learning (TRL[5]) library by von Werra et al. (2020), as it is directly integrated with Transformers. For each benchmark and model size, I train a new model and save it to disk. For benchmarking, this saved model is then loaded in, taking special care to load the CCA parameters with the "state dict" approach that Transformers uses. Unfortunately, the resources required to train Llama-2 70B exceed the capacities of the workstation, even when only CCA parameters are unfrozen and when loading the model using data types that occupy less memory. As such, I fine-tune BioLlama 7B and 13B on each benchmark individually. Table 3.1 shows the used hyperparameters. My fine-tuning approach directly follows a tutorial by Weights & Biases[6].

Table 3.1: *The hyperparameters used to fine-tune BioLlama CCA parameters using TRL's Supervised Fine-Tuning trainer (SFTTrainer).*

Hyperparameter	**Chosen value**
Learning rate	2e-4
Learning rate scheduler type	Cosine
Batch size	2
Max. steps for BioASQ5b	486
Max. steps for PubMedQA	1000
Max. steps for MedQA-4 and -5	10,178
Max. steps for MedMCQA	100,000
Warmup steps	Max steps // 10
Gradient accumulation steps	1
Trainer type	TRL SFTTrainer
SFTTrainer packing	True
SFTTrainer max. sequence length	1024

[5]https://github.com/huggingface/trl

[6]https://wandb.ai/capecape/alpaca_ft/reports/How-to-implement-fine-tuning-of-an-LLM-Part-1-Dataset-for-Instruction-Tuning--Vmlldzo1NTcxNzE2

4 Results & Evaluation

This section analyses and discusses the performance of Llama-2 and BioLlama on my chosen biomedical benchmarks across varying sizes and settings, with each subsection consisting of a response to a research question. Note that setting the generation temperature value to 0.01 essentially renders outputs deterministic. As there is no potential variability in my results, I do not include measures of uncertainty such as error bars as they would be neither meaningful nor appropriate given this context.

4.1 Responding to my research questions

4.1.1 RQ1. How does few-shot learning affect Llama-2's accuracy on biomedical benchmarks?

Figure 4.1 showcases the results of my experiments to investigate the impact of few-shot learning. For each benchmark, I use the same "few-shot" example across the model sizes to ensure fair comparisons.

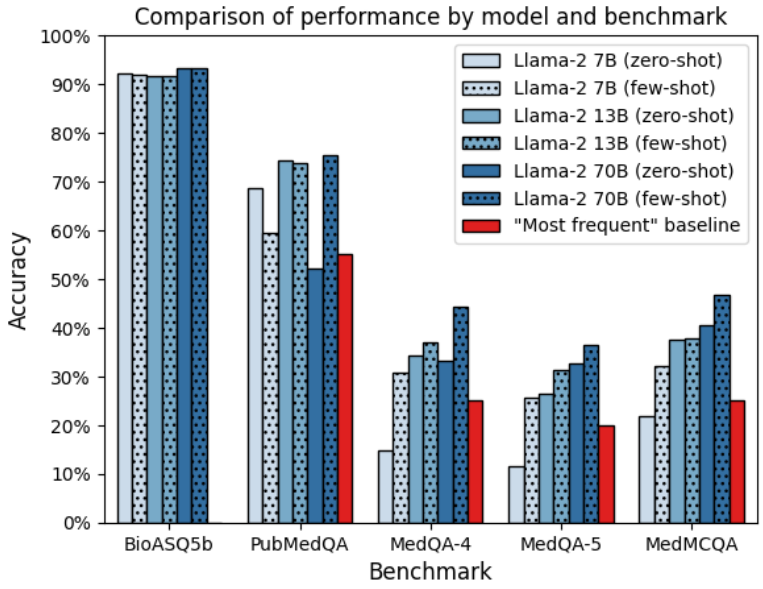


Figure 4.1: *A graph contrasting Llama-2's accuracy on biomedical benchmarks on zero-shot and few-shot settings across different sizes. While the effect seems negligible on BioASQ and different for varying model sizes on PubMedQA, it leads to pronounced gains on all OpenQA tasks. BioASQ5b has no dummy baseline, as it is a factoid benchmark where "random guessing" does not make much sense.*

In my results, I observe that few-shot learning has a different impact on QA tasks than it does on OpenQA tasks: On BioASQ5b, few-shot learning shows almost no impact on Llama-2's

performance across all model sizes. Meanwhile, when using PubMedQA as a benchmark, few-shot learning leads to a degradation in performance on 7B, a roughly equal performance on 13B and a substantial increase in performance on 70B.

The original motivation of few-shot learning in LLMs as discussed by Brown et al. (2020) is to teach the model how to perform a task it has not seen in its training data, by inserting examples into its context window. Therefore, the contrasting impact of few-shot learning across QA and OpenQA benchmarks may primarily be reflective of their different nature: BioASQ5b is an extractive task where the model needs to identify the right factoid to answer a question from a number of snippets. The unchanged performance of Llama-2 when employing few-shot learning may suggest that an extractive task of this nature could have been present in Llama-2's training data. Thus it is possible that the accuracy score cannot be improved through this approach as its performance potential is already exhausted. Meanwhile, PubMedQA follows the same extractive nature but requires yes/no/maybe outputs instead of factoids, and I observe that larger parameter counts are necessary for few-shot learning to help and not harm model performance. This again might mirror a nuance against BioASQ5b, where the model really only needs to copy a factoid from the context. PubMedQA requires the model to go a step further and perform reasoning, which has shown to benefit from more parameters.

Results obtained by benchmarking Llama-2 on OpenQA benchmarks paint a different picture entirely. Here I find that few-shot learning is universally helpful across different benchmarks and model sizes. While the gains seem more pronounced at lower parameter counts (few-shot learning raises Llama-2 7B's accuracy on MedQA-5 from 11.5 to 25.6%) even Llama-2 70B can benefit from it. I find this also to be heavily related to Llama-2's ability to follow instructions on how to generate its outputs: When shown an example of how to structure a response, Llama-2 is much more likely to follow the required scheme. This indicates that the "benchmark accuracy" as extracted by my pipeline may not be directly indicative of the actual biomedical knowledge the model possesses, but confounded by its ability to follow the structure I require.

```
"Position of vocal cord in cadaver is:\n (1) Median\n (2) Paramedian\n (3) Intermediate\n (4) Full Abduction",
"3",
"The position of vocal cords in a cadaver is: (3) Intermediate."

"Organisms that has not been cultured successfully so far is-\n (1) Leptospira\n (2) Treponema pallidum\n (3) Bordetella\n (4) Staphylococcus",
"2",
"Treponema pallidum"
```

Figure 4.2: *Llama-2 7B's responses to two MedMCQA questions under a zero-shot setting. For each question, the first line shows the question and answer options, the second line the mark scheme and the third line Llama-2 7B's output.*

```
"Position of vocal cord in cadaver is:\n (1) Median\n (2) Paramedian\n (3) Intermediate\n (4) Full Abduction",
"3",
" 3"

"Organisms that has not been cultured successfully so far is-\n (1) Leptospira\n (2) Treponema pallidum\n (3) Bordetella\n (4) Staphylococcus",
"2",
" 2"
```

Figure 4.3: *Llama-2 7B's responses to the same MedMCQA questions under a few-shot setting. This time, it more consistently structures its response as instructed. Note that the actual prompt the model sees contains the few-shot examples and instructions on how to format its output.*

Figures 4.2 and 4.3 demonstrate this effect in Llama-2 7B's responses to MedMCQA questions. In these cherry-picked examples, the zero-shot response is factually correct but not in the format as required, leading the EM score to underestimate model performance. In a few-shot setting, Llama-2 7B continues to respond correctly and this time formats its answers as it is prompted to. Through inspecting benchmark outputs manually, I also find responses to few-shot prompts more likely to be factually accurate.

Finally, Figure 4.1 also displays baseline performances for every benchmark. As BioASQ5b is a factoid QA benchmark where every response is structured slightly differently, a baseline does not make sense. Conversely, all other benchmarks are multiple-choice, so it is important to ensure they outperform a "most frequent" or randomized guessing strategy. My findings show that especially under a zero-shot regime, Llama-2 can struggle to improve upon those dummy thresholds and in some instances even remains below them. However, when applying few-shot learning, Llama-2 exceeds the respective baseline on every benchmark, even in its smallest 7B variant.

Therefore, with reference to RQ1, these findings lead me to conclude that few-shot learning can be a powerful tool to attain consistent model outputs that more frequently answer biomedical questions correctly. Motivated by this, I use this few-shot setting uniformly throughout the rest of my investigation to reduce the chance of Llama-2 or BioLlama generating a response that is factually correct but not recognized as such by my pipeline.

4.1.2 RQ2. What is the impact of RETRO-fitting Llama-2 on its ability to answer biomedical questions without fine-tuning?

To answer this question, I RETRO-fit Llama-2 to produce BioLlama in all 7B, 13B and 70B size variants, following the method set out in Section 3.2. At inference, BioLlama retrieves from RCT200k using MedCPT for all benchmarks. Figure 4.4 contrasts its performance to that of Llama-2 on my chosen biomedical benchmarks, using the same few-shot prompts for both models.

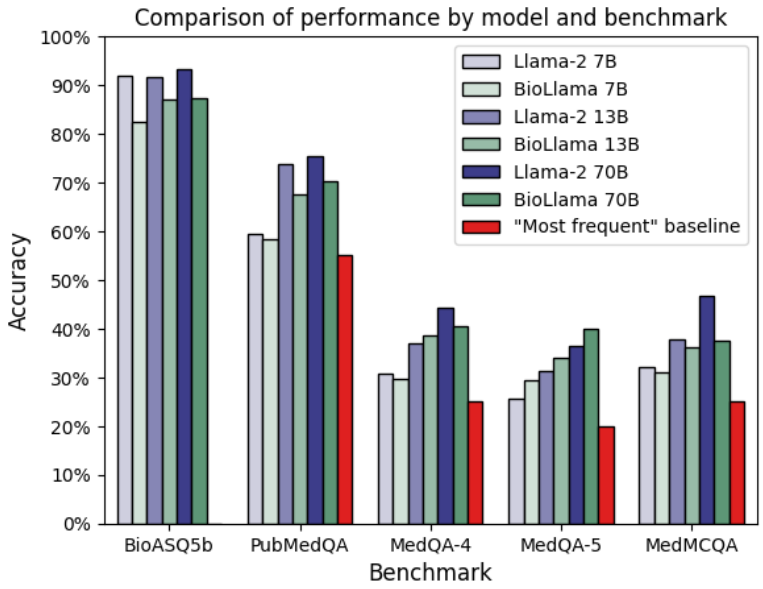


Figure 4.4: *A graph comparing BioLlama's performance on biomedical benchmarks to the Llama-2 baseline set in 4.1.1. Aside from anomalous results on MedQA, BioLlama's performance indicates a consistent degradation in benchmark accuracy when CCA parameters are randomly initialized and not training.*

I find that generally speaking, RETRO-fitting BioLlama with CCA parameters that are randomly initialized and not trained deteriorates the models ability to correctly respond to questions. It does not seem to lead to a complete breakdown in model generations, as can be observed in PubMedQA results where accuracy decreases only by 1-6%. Further, BioLlama continues to outperform the

randomized/"most-frequent" guessing thresholds on all multiple-choice benchmarks in all model sizes. Nevertheless, Figure 4.4 demonstrates a clear decline in model capabilities.

When considering the modification that is performed, these results are to be expected. Llama-2 has been trained on 2 trillion tokens (Touvron et al. 2023b), over which each of its parameters have continually been carefully adjusted. RETRO-fitting in this way, where new modules are added but contain completely random weights, simply introduces noise. Taking the example of BioLlama 7B, the difference to Llama-2 7B is that BioLlama adds CCA parameters at layer 16. As the hidden states move through the model, they then have a transformation applied to them at layer 16 that is completely random. While layer 17 "expects" to be passed hidden states that are at a certain point in decoding, it instead receives them slightly differently from what it has been tuned for. Notably, in each BioLlama variant only one middle layer is RETRO-fitted as outlined at the end of Section 3.2. So while this introduces noise, the rest of the layers (the remaining 31, 39 and 79 layers of Llama-2 in its 7B, 13B and 70B variants that remain untouched) still perform as normal. This means that while performance is indeed degraded slightly, the overall ability of the model to respond to questions remains intact. An example of this can be seen in Figure 4.5.

```
"Chronic functional somatic symptoms: a single syndrome?",
"no",
"no"

"Actinobaculum schaalii, a cause of urinary tract infections in children?",
"maybe",
" Yes</ANIPT>\n\nWhat is the number of urine"
```

Figure 4.5: *BioLlama 13B's responses to two PubMedQA questions. The format again follows the question/mark scheme/answer format. As can be seen, some tokens produced by the model can seem incoherent. Again, note that the actual prompt that was used is much longer than just the shown question, as it contains the relevant context snippets as well as instructions on how to format responses.*

In the first example, BioLlama continues to perform as normal. However, the second example directly showcases the potentially detrimental impact of even introducing a small amount of noise. Here, the model has followed its response with "</ANIPT>" instead of "</ANSWER>" as required. Consequently, the regex function in the pipeline fails to extract the model response that is usually situated between the "<ANSWER>...</ANSWER>" pairs, as no closing "</ANSWER>" tag was found. The tokens that follow are as such hallucinated and nonsensical. "</ANIPT>" is an example of how the noise injected into the model through randomized RETRO modules can corrupt model generations. While additional checks after a regex failure attempt to still extract a response to the question from what has been generated, in this case even the response "Yes" would be incorrect. Returning to RQ2, I find that the impact of RETRO-fitting Llama-2 without further fine-tuning of the new parameters leads to a clear diminishing of the models ability to answer biomedical questions. The added modules introduce noise and corrupt next-token-predictions. This makes it abundantly clear that they have to be trained, as in the absence of fine-tuning they damage the model's capabilities instead of enhancing them. These findings directly motivate my choices in training the CCA parameters, as investigated in the next section.

4.1.3 RQ3. Can supervised fine-tuning of BioLlama's CCA parameters improve its performance on biomedical benchmarks?

To combat the issues described above, I employ TRL's supervised fine-tuning trainer to fine-tune BioLlama's CCA modules on my biomedical benchmarks as outlined in Section 3.3. Notably, all of the original Llama-2 parameters remain frozen, and I only enable gradient updates to the new CCA parameters. Unfortunately, the computational requirements of a 70B model exceed my resource capacities, so I only fine-tune 7B and 13B variants. The performance of BioLlama after fine-tuning is shown in Figure 4.6.

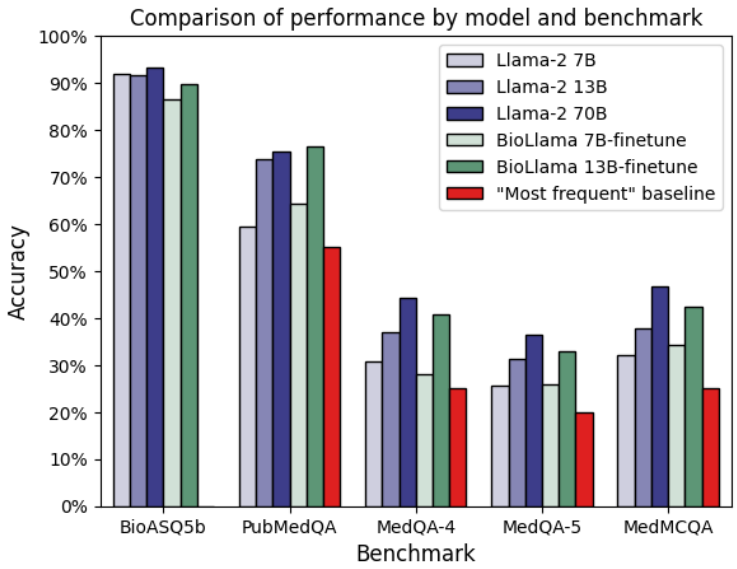


Figure 4.6: *A graph comparing BioLlama's performance on biomedical tasks against Llama-2 and the dummy baselines, after its CCA parameters were fine-tuned. As previously mentioned, few-shot learning is used in all prompts to achieve consistent model outputs.*

I find that fine-tuning not only remedies the performance degradation observed in RQ2, but allows BioLlama to exceed the Llama-2 baselines across all model sizes. In the absence of BioLlama 70B, BioLlama 13B exhibits strong performance across the board. It consistently exceeds Llama-2 13B, and on PubMedQA even outperforms Llama-2 70B which is ~5.38x its size.

This is suggestive of the notion that RETRO-fitting followed by fine-tuning of the new parameters enables really substantial gains on biomedical benchmarks. Within the wider context of retrieval-augmented generation, it provides further evidence that LLMs can benefit from gaining access to relevant context snippets at inference time. Furthermore, it stays consistent with findings by Frisoni et al. (2022), furthering the hypothesis that RETRO-fitting is a lucrative method of retrieval augmentation which deserves further investigation. Performing cross-attention on text other than the input seems like a powerful approach to steering output generation, and these results demonstrate its power in biomedical contexts. RCT200k is a comparatively small corpus, and the fine-tuning involved was completed relatively quickly (between ~2 and ~12 hours, depending on the size of the training set for each benchmark). It is plausible that larger retrieval corpora, more comprehensive training sets and an increase in the number of layers that are RETRO-fitted can enable even further performance increases.

In addition, it also helps motivate fine-tuning CCA parameters in the first place. It may be argued that training some of BioLlama's parameters on a task's training set and then benchmarking it on the respective test set comprises an unfair advantage over the Llama-2 baselines. While it is true that Llama-2 may not have seen the relevant dataset at all during its pre-training process, BioLlama and Llama-2 are, to a large extent, entirely identical. For example, of BioLlama 13B's 13,120.73M parameters, 268.70M become trainable, which is a mere ~5% of its total weights. Hence, it is notable that such a comparatively small difference to Llama-2 has enabled very consistent performance increases. Fine-tuning the new modules allows them to contribute towards more reliably correct outputs, instead of injecting noise into the decoding process. To illustrate this with qualitative results, Figure 4.7 shows the responses of BioLlama 13B to the same PubMedQA prompts as in 4.5.

```
"Chronic functional somatic symptoms: a single syndrome?",
"no",
" no"

"Actinobaculum schaalii, a cause of urinary tract infections in children?",
"maybe",
" yes"
```

Figure 4.7: *BioLlama 13B's responses to the same PubMedQA questions as in Figure 4.5. As can be seen, model outputs become much more consistent, though incorrect responses are not eliminated.*

In these examples, BioLlama has structured its answers as instructed. I find this to be the case for the vast majority of responses, which suggests that fine-tuning transforms the parameters from distorting generations to making them more consistent and reliable.

Furthermore, we can dig deeper and look specifically at the retrieved text that BioLlama is performing chunked cross-attention on to get a better understanding for why it performs so well. Figure 4.8 shows BioLlama 13B's response to a MedQA-5 prompt alongside the texts that have been retrieved by MedCPT.

```
MedCPT retrieved the following chunks:
  1 - ['infant death during the first week of life ( or = 0. 88, 95 % ci = 0. 81 - 0. 95 ). infants of women who']
  2 - ['assessed the probable cause of sudden death and evaluated how these causes varied with time after mi. of 1
05 deaths considered sudden on clinical grounds, autopsy suggested the following causes']
  3 - ["individual educational session in the maternity ward about infant sleep position significantly increased t
he prevalence of supine position for sleeping in the infant's third month. nevertheless, the intervention"]
  4 - ['kc daily between days of life 5 to 10 or to standard incubator care. infants were monitored for bradycardi
a ( heart rate < 80 ) or oxygen desaturation ( <']
  5 - ['potentially more acceptable position monitor and supine - avoidance device for managing supine - predomina
nt osa and snoring. in - laboratory evaluation of position recording accuracy versus video recordings (']
***Generating***
<QUESTION>A 3-month-old baby died suddenly at night while asleep. His mother noticed that he had died only after s
he awoke in the morning. No cause of death was determined based on the autopsy. Which of the following precautions
 could have prevented the death of the baby?
 (A) Placing the infant in a supine position on a firm mattress while sleeping
 (B) Routine postnatal electrocardiogram (ECG)
 (C) Keeping the infant covered and maintaining a high room temperature
 (D) Application of a device to maintain the sleeping position
 (E) Avoiding pacifier use during sleep</QUESTION>
<ANSWER>  (A) Placing the infant in a supine position on a firm mattress while sleeping</ANSWER>
```

Figure 4.8: *The 5 chunks that were retrieved by MedCPT, followed by a MedQA-5 question, and then after the <ANSWER> tag, BioLlama 13B's response. The correct answer is (A), ie. to place the infant in a supine position, which BioLlama has correctly identified. Mentions of supine sleeping positions are highlighted in red.*

As is visible in the Figure, the retrieved chunks 1, 2 and 4 are broadly relevant to the question but do not directly contain the right answer. Meanwhile, chunks 3 and 5 explicitly mention supine sleeping positions, as highlighted in red. Through the added RETRO parameters, BioLlama has performed cross-attention on all these chunks, and it is plausible that the mentions of supine sleeping positions have helped guide it towards the correct response.

This is the promise of RETRO at work. It enables the transformer to pay attention to relevant snippets retrieved from a database. In this case, the domain-adapted MedCPT retriever has done an excellent job at returning text from RCT200k that is not only relevant to the question, but helps BioLlama identify the right answer. Both of these components are modular: MedCPT could be replaced with another retriever when considering a different domain. Similarly, the texts in RCT200k can individually be modified if new studies reveal that specific findings in it need to be updated. This is a profound research finding: If a reliable and high-quality database is paired with an very capable retriever, a language model can incorporate the returned texts to make it more performant. Figure 4.8 also shows how the model becomes more explainable. We

can trace a connection between the snippets the model attends to and the answer it produces as a result. With further research, this can help make RETRO models less opaque.

However, these results also come with important caveats. Firstly, Figure 4.8 is a cherry-picked example in which the right text was contained in RCT200k, the right snippets were returned by MedCPT and the appropriate transformation was performed by BioLlama to incorporate this. All three of these factors need to line up in order to steer the language model. There were many cases on all benchmarks where a single point of failure deteriorated the downstream performance. This is also evident in the second question in Figure 4.7, where BioLlama follows the required output scheme but still answers it incorrectly. This is an important observation and is representative of BioLlama's performance on the whole. While the process of RETRO-fitting and fine-tuning has enabled considerable enhancements in model capabilities, it has far from fully remedied model hallucinations. Returning to MedQA-5, BioLlama 13B answers a mere 32.9% of the questions correctly. It is important to contextualise the idea that a neural network getting Medical Licensing Exam questions right with relative consistency as shown here was unthinkable some ~5 years ago. However, BioLlama also answers questions incorrectly a large proportion of the time, remaining below 50% accuracy on all OpenQA benchmarks. In contrast to QA tasks, the model cannot extract a correct response from context snippets, so these benchmarks come closer to testing what biomedical knowledge the model possesses. While expecting the model to get every question right on every OpenQA benchmark would be unrealistic, the relatively low accuracy scores re-affirm concerns surrounding LLMs correctly answering biomedical questions with any kind of consistency. Even though RETRO-fitting takes an important step in the right direction, it mostly fails in mitigating hallucinations.

Meanwhile, BioLlama exhibits a much higher degree of reliability on QA tasks. Though BioLlama 13B obtains a slightly lower score than Llama-2 13B on BioASQ5b, the difference amounts to a mere ~2%, and both performances are roughly within 10% of achieving full marks. On PubMedQA, BioLlama considerably outperforms Llama-2 across model sizes. While the top score of 76.5% that BioLlama 13B achieves is still far from perfect, it is important to note that this score is very close to the human performance of 78% as reported by Jin et al. (2019). This exemplifies that even at 13B parameters, BioLlama demonstrates a formidable capability in performing biomedical reasoning.

Finally, we can consider the training loss of BioLlama in its varying sizes and on the varying benchmarks as a proxy for the model's ability to incorporate chunks from external literature when tested on a downstream task. Figure 4.9 shows how the loss decreases for the OpenQA tasks (loss curves on the two QA tasks are included in Appendix B).

This visualisation reveals nuances in how training affects BioLlama differently depending on its size variant. On these OpenQA benchmarks, the training loss of BioLlama 13B is consistently much lower than that of BioLlama 7B, indicating that even if only one layer is RETRO-fitted, its greater total parameter count may enable it to better adapt its new modules to the downstream task. This graph only shows performance up to 20k training steps to be able to better see the MedQA curves that finish after 10k steps. Nonetheless, BioLlama 7B's training loss on MedMCQA does continue to decrease as it approaches the total of 100k training steps, seemingly levelling out around a training loss of ~0.55. Simultaneously, BioLlama 13B reaches a minimum training loss of ~0.37 relatively early into fine-tuning and continues to hover there for the rest of the training steps. This again is indicative of BioLlama 13B's advantage over BioLlama 7B purely as a result of its larger parameter count. Interestingly, this faster reduction in training loss seems correlated to benchmark scores too: As shown in Figure 4.6, on all OpenQA tasks BioLlama 13B improves upon Llama-2 13B's scores much more than BioLlama 7B improves upon Llama-2 7B. Therefore, it can be concluded that one of the many advantages of larger parameter counts is also an improved ability to adapt to modifications such as RETRO-fitting.

Returning back to RQ3, I find that supervised fine-tuning of the added CCA parameters enables

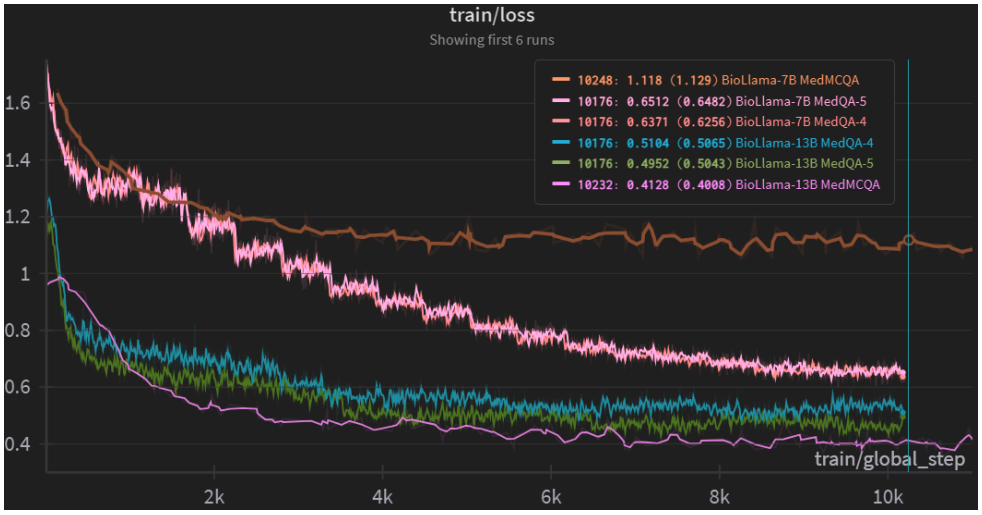


Figure 4.9: *A graph showing the loss curves of BioLlama 7B and 13B on MedQA-4, MedQA-5 and MedMCQA. As these benchmarks have much more training samples, training runs for longer. In this context, BioLlama 13B seems to be much better at exploiting its larger parameter count, as exemplified by loss curves consistently being lower compared to BioLlama 7B on every benchmark.*

BioLlama to surpass the baseline scores set by Llama-2 on most tasks. It remedies the issues of randomised noise explored in RQ2 and in some cases results in substantially improved benchmark scores. This is an exciting research finding, as it provides evidence to suggest that RETRO-fitting a decoder-only model can make it more reliable.

4.1.4 RQ4. Can BioLlama benefit from supervised fine-tuning beyond one epoch?

In Section 3.3 I list my training steps as 486 for BioASQ5b, 1,000 for PubMedQA, 10,178 for MedQA-4 and -5, and 100,000 for MedMCQA. These values are chosen as these are the respective sizes of the training sets I employ, implying that in RQ3 I train for 1 epoch only. This approach means that the model sees every training example only once, and helps to prevent overfitting.

However, similar investigations have shown that apparent downsides of training for more than 1 epoch can be outweighed by surprising benefits. For example, Ouyang et al. (2022) train InstructGPT for 16 epochs, finding that training beyond 1 epoch exhibits over-fitting on the validation loss in similar observations as Wu et al. (2021). Nonetheless, they discover that human preference ratings and the scores of their Reward Model continue to increase as the model trains for longer. Similarly, Zhou et al. (2023) find that more training steps can lead to higher perplexity on the validation step, as often is indicative of over-fitting. Nevertheless, the quality of generations seems to increase, further supporting multi-epoch training. In this same vein, Taori et al. (2023) fine-tune a Llama-1 model (Touvron et al. 2023a) and use 3 epochs for its 7B and 5 epochs for its 13B variant, demonstrating strong performances. This shows that prior paradigms of 1-epoch fine-tuning of LLMs may be up for debate, and that existing metrics such as validation loss are potentially not the only indicators that should be considered to judge over-fitting.

Finally, in the investigation that is the closest to mine, Frisoni et al. (2022) fine-tune BioReader for 30 epochs. While their approach of unfreezing all layers during fine-tuning contrasts to my unfreezing of only RETRO parameters, it is still notable that their chosen number of epochs is so high. Unfortunately, they do not justify this choice further or provide quantitative analyses on the impact of different epoch numbers. However, with 1-epoch training durations of up

to 12 hours, I am limited from scaling to such training lengths anyways. Therefore, I instead perform experiments on fine-tuning for 2 epochs by doubling the number of training steps for every benchmark, forcing the training dataloader to start loading examples from the start of the dataset again once its end is reached. After the 2-epoch fine-tuning, the produced BioLlama variants are benchmarked in the same way as all other models. The respective results are shown in Figure 4.10.

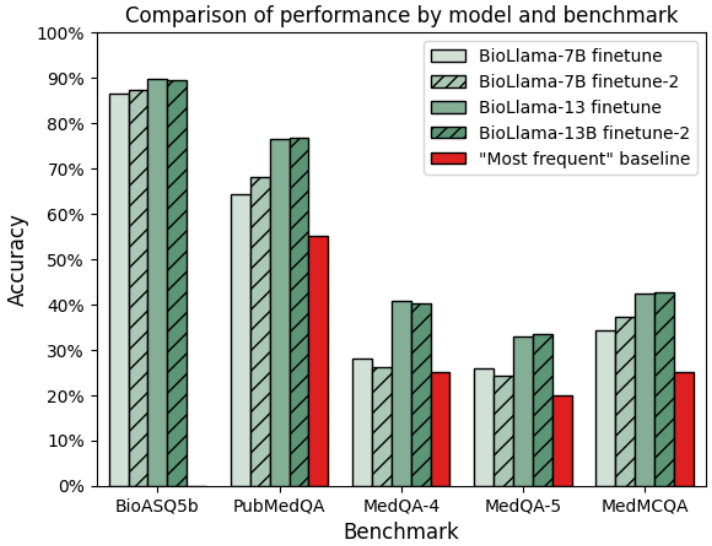


Figure 4.10: *A graph comparing the benchmark results of BioLlama under 1-epoch and 2-epoch fine-tuning regimes. Note that in the legend, "finetune-2" indicates 2-epoch training. While 2-epoch training seems to yield very minor improvements on some benchmarks, its effect generally seems to be negligible.*

As becomes evident in the graph, it does not appear that 2-epoch training is a particularly viable approach to achieve further improvements on any of the benchmarks. This is a regrettable result, as the previously discussed investigations support the hypothesis that training beyond 1 epoch can enable further improvements. However, it is also a valuable finding in indicating that lower training step numbers are sufficient for tuning new RETRO parameters.

Returning to the original motivation for fine-tuning, it is used in this investigation to teach the model how to use its new parameters to incorporate information in chunks retrieved from the literature. While it is conceivable that intelligent training approaches can derive continuous benefits from prolonged fine-tuning of only the new RETRO modules, it is also important to consider that they are only a small proportion of the total model size. Therefore, these findings may indicate that the added parameters can only be tuned "so much", and that when a local minimum is found, additional fine-tuning does not really benefit the model further. There seem to be slight variations depending on the benchmark: In the case of the MedQA scores, further training seems to have slightly degraded BioLlama's performance in its 7B and 13B variants. Meanwhile, on PubMedQA, further fine-tuning seems to have improved performance slightly, enabling a new high-score of 76.7% by BioLlama-13B after 2-epoch fine-tuning.

While these results may be indicative of an upper limit to the utility of further fine-tuning of CCA parameters, it leaves the question of fully-fledged fine-tuning of BioLlama unanswered. In this setting, all layers are unfrozen and allowed to adjust their parameters to decrease the training loss further. This is what Frisoni et al. (2022) employ to enable their highest downstream gains, and it can be theorized that this would be highly beneficial for BioLlama too. It is also important to note that Frisoni et al. (2022) fine-tune one unified BioReader model on all tasks

simultaneously. Unfortunately, they do not provide sufficient details for me to recreate this, so I train a new model for every benchmark. This further limits the degree to which BioLlama and BioReader can be compared. In order to limit the computational strain and investigational scope of this project, both unified fine-tuning as well as all-layer fine-tuning are left as directions for future work.

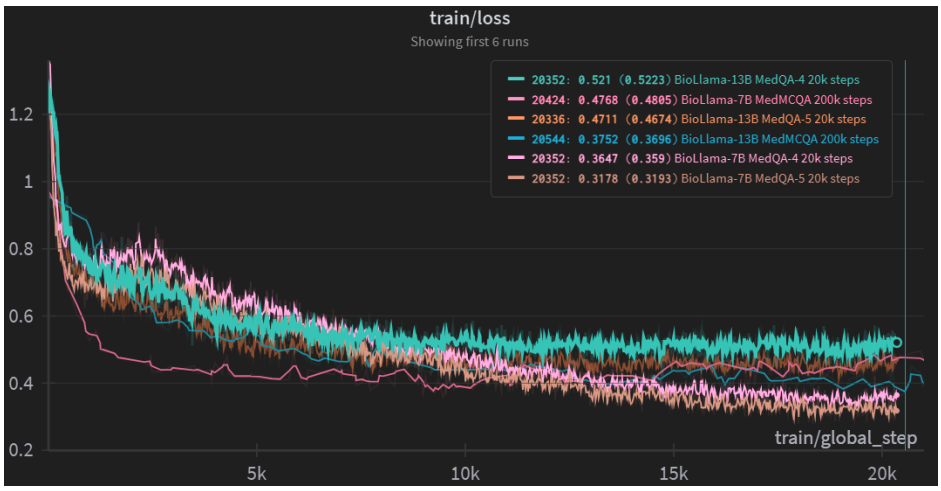


Figure 4.11: *A graph displaying the loss curves of BioLlama 7B and 13B on the OpenQA benchmarks over 2 epochs. I apply running average smoothing on all loss curves to prevent them from overlapping and hiding each other.*

Finally, we can also consider the training loss of BioLlama on 2-epoch training. The respective visualisations for the OpenQA benchmarks are shown in Figure 4.11. Curiously, I find that BioLlama 7B finishes fine-tuning with a lower training loss than BioLlama 13B. Again, the graph only shows curves up to ~20k steps so that the MedQA curves are visible, but for MedMCQA the curves continue up to 200k steps. Here I observe this trend too: BioLlama 13B seems to find a local minimum around a loss of ~0.38 quickly, while BioLlama 7B's training loss continues to decrease over the whole training run and finishes at a lower value of ~0.21. These results are particularly fascinating as they seemingly do not to translate into downstream benchmark performance. While BioLlama 7B's training loss on MedQA-4/5 after 2 epochs is considerably lower than that of BioLlama 13B, it has lower benchmark scores than BioLlama 7B after only 1 epoch. In minor experimentation, I tried to manipulate hyperparameters and looked at generated outputs manually to gain a better understanding of this effect but could not identify further reasons. This too could be an interesting direction for future work.

To conclude, with reference to RQ4 I do not find that further fine-tuning has a substantial impact on BioLlama's performance on biomedical tasks. It seems that 1 epoch is sufficient for the model to perform the necessary adaptation during training.

4.1.5 RQ5. How does BioLlama's performance compare to the Llama-2 baselines when considering different biomedical subjects?

In addition to the previous in-depth analyses of BioLlama's performance across different sizes, benchmarks and epochs, I also wanted to take a closer look at how it tackles the benchmarks individually. This is motivated in part by the fact that relying on only accuracy scores in machine learning can be dangerous as such summary statistics are often very reductionist (Olah and Jermyn 2024). For example, if an LLM employed in a healthcare setting achieves 97% accuracy because it answers the vast majority of easy questions correctly, this metric can obscure the fact that the 3%

of incorrectly answered questions actually have the largest impact on patient outcomes. Motivated by this, I endeavour to provide a more detailed breakdown of BioLlama's benchmark performance. I utilize its responses to MedMCQA questions for this, as every MedMCQA questions is tagged with a "subject_name" field. This allows me to take a closer look at how BioLlama 13B (my largest RETRO-fitted model) performs in different subject areas in comparison to Llama-2 13B. The results of this are displayed in Figure 4.12.

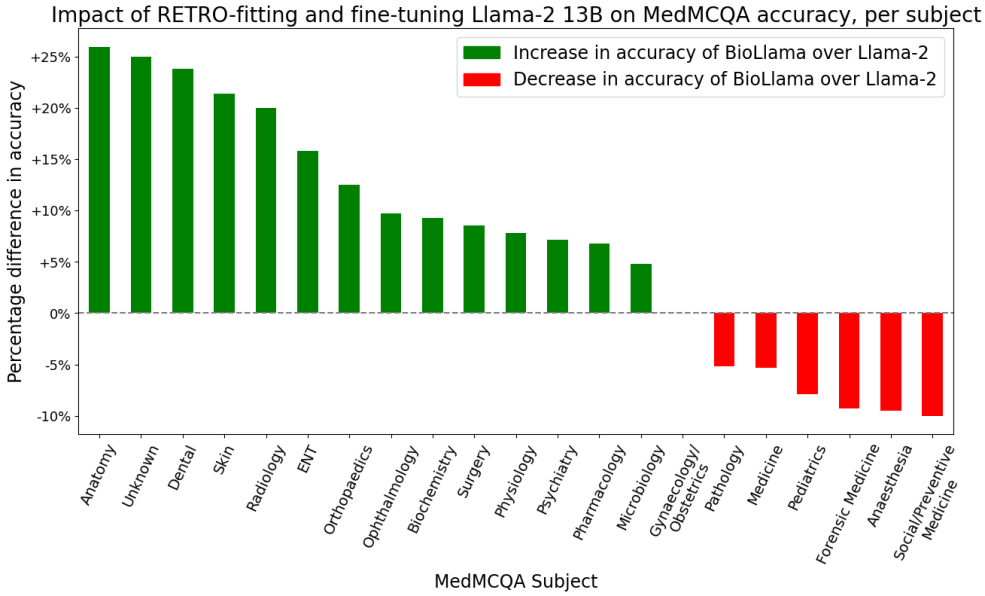


Figure 4.12: *A graph displaying the percentage difference in accuracy between BioLlama 13B (1-epoch) and Llama-2 13B on different MedMCQA subjects, sorted by the magnitude of the increase. Improvements in accuracy of BioLlama over Llama-2 are shown in green, decreases in red.*

As BioLlama 13B attains an accuracy score of 42.3 and Llama-2 13B achieves 37.8, it is expected for BioLlama to outperform Llama-2 on most subjects, which is evidently the case. What is fascinating about this graph is the nuances in performance differences for different subjects. For better interpretations of these results, I consulted a medical doctor to be able to incorporate feedback from a domain expert. Informed by this discussion, I propose the following hypothesis:

BioLlama achieves decent accuracy scores on "science-oriented" subjects of pre-clinical medicine such as **Anatomy**, **Biochemistry** and **Physiology**. This may indicate that the "Background" sections of the abstracts contained in RCT200k contain more factoids that are relevant to these questions. Further, BioLlama consistently outperforms Llama-2's baselines on so-called "smaller" subjects such as **Dental** or **Skin**. Again, it is possible that RCT200k contained many randomized control trials that focus on treatment of diseases which those MedMCQA questions ask about.

On the contrary, the larger subjects such as **Medicine** and **Pathology** are more challenging for BioLlama. These subjects are made up of a larger number of sub-specialties in which different methodologies may apply. It is possible that Llama-2's pre-training corpus already contained a lot of information on these subjects. Unfortunately, the exact details of what Llama-2 was trained on were never published, so one can only speculate that web-scrapes may have included large health websites such as healthline.com and webmd.com. In contrast to this, it is possible that RETRO-fitting on RCT200k injects more noise than it helps.

While the specific data that Llama-2 was trained on is not accessible, RCT200k is. Every abstract in the dataset is referenced with its PMID (which is essentially a unique identifier that PubMed has associated with a given paper), which I can use to query Entrez. Entrez is an E-utility through

which PubMed can be queried electronically, and is integrated into the biopython library. This allows me to compile a list of all PMIDs in RCT200k and then query PubMed with Entrez to gain more information about each abstract. Among many other details, this gives me access to the associated MeSH terms. MeSH stands for Medical Subject Headings and is a vocabulary thesaurus to help index PubMed entries. Querying the PubMed API on a random sub-sample of ~35% of RCT200k PMIDs for 13 hours, I compile statistics on the most common "Qualifier Names" in RCT200k's MeshHeadings, as shown in Figure 4.13.

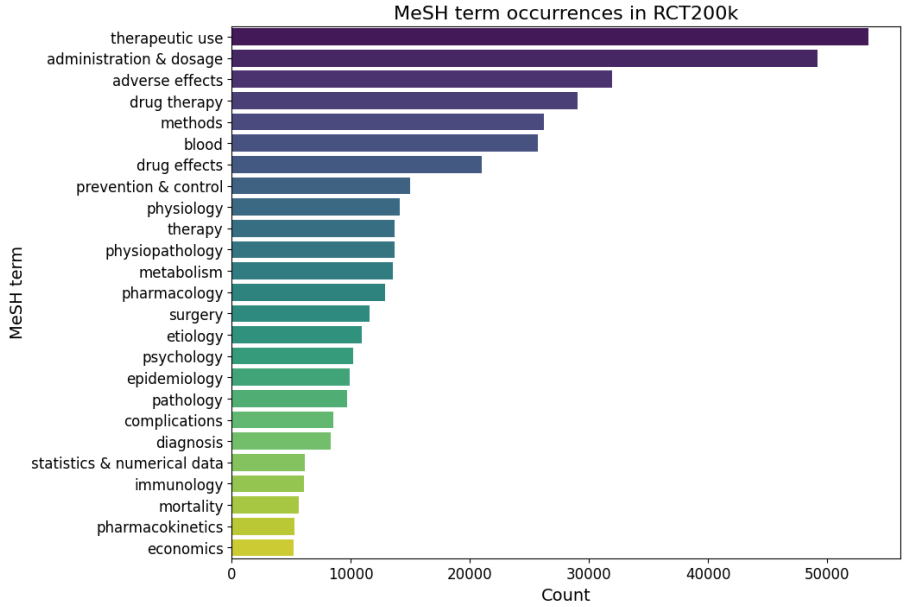


Figure 4.13: *A graph showing the 25 most common MeSH qualifier names in RCT200k.*

The results are in line with what one might expect from an RCT corpus: Terms such as "therapeutic use" or "administration & dosage" appear frequently, which is coherent with the fact that RCT200k contains texts describing drug trials. The terms "blood" and "physiology" also occur very frequently, and abstracts concerning them may be particularly helpful for MedMCQA questions on **Physiology**, where BioLlama outperforms Llama-2. Similarly, many RCT200k abstracts are tagged with "metabolism" and arguably could be helpful for **Biochemistry** questions, on which BioLlama again exceeds Llama-2 baselines. In the same vein, papers on "pharmacology/pharmacokinetics", "psychology" and "immunology" may have contained relevant factoids for questions on **pharmacology**, **psychiatry** and **microbiology** respectively, all of which exhibit stronger BioLlama scores than Llama-2 scores.

However, it must be emphasized that these suggested reasons remain speculative. For example, papers tagged with "physiopathology" also come up frequently in RCT200k, and yet BioLlama has a lower accuracy score on **pathology** questions than Llama-2. Furthermore, this analysis does not indicate whether MedCPT actually successfully *retrieves* the papers with these tags for the relevant questions. As such, the true reasons for performance differences would require an even more in-depth analysis of individual questions, posing another direction for future research.

Referring back to RQ5, I find that the impact of RETRO-fitting is different across different subject types on the MedMCQA benchmark. It appears that performance on questions in subjects of which there are many papers in the retrieval corpus are particularly benefited, though further research would be necessary to confirm this finding.

4.1.6 RQ6. How does BioLlama's performance compare to other general-purpose and domain-specific models?

Finally, it is important to evaluate BioLlama's performance not only in relation to the Llama-2 baselines, but also other general-purpose and biomedicine-specific models. Table 4.1 shows a comprehensive comparison to the aforementioned BioReader, which was never open-sourced but is domain-adapted, as well as Med-PaLM 1/2 and MedPrompt, which are proprietary and domain-adapted. In addition, I compare to GPT-4, which is proprietary and general-purpose.

Table 4.1: *A comparison of BioLlama against other models in the field. Strongest scores are in bold, and all results aside from BioLlama are taken from the papers that report on them. BioLlama 13B's scores are those under the best setting for each respective benchmark. While BioLlama outperforms BioReader on BioASQ5b and GPT-4 on PubMedQA, it lags noticeably behind on all other benchmarks.*

Model	Size	BioASQ5b	PubMedQA	MedQA-4	MedQA-5	MedMCQA
BioReader	229M	81.88	-	43.0	-	-
Med-PaLM	540B	-	79.0	67.6	-	57.6
Med-PaLM 2	540B	-	81.8	86.5	-	72.3
GPT-4	?	-	74.4	81.4	78.6	72.4
MedPrompt	?	-	**82.0**	**90.2**	-	**79.1**
BioLlama 13B	13B	**89.79**	76.7	40.8	32.9	42.7

By considering the performance of other models, BioLlama's strengths and weaknesses become more apparent. Notably, on both extractive QA tasks, BioLlama stays very competitive. On BioASQ5b, BioLlama is able to outperform BioReader considerably, further exemplifying the strength of decoder-only RETRO-fitting. However, this result comes with a caveat: BioLlama 13B is >56x larger than BioReader. The difference in their scores is therefore more of a testiment to the power of BioReader. Perhaps the fact that it RETRO-fits 3 layers, pre-trains on biomedical literature or fine-tunes all layers on the benchmarks yield this impressive parameter efficiency.

On PubMedQA, BioLlama is outperformed by all other models, with one exception: GPT-4. BioLlama 13B is able to exceed GPT-4's accuracy on PubMedQA by 2.3%. This is a very exciting research result, as GPT-4 has remained a very dominant model across almost all benchmarks since its release more than a year ago. The fact that BioLlama is capable of answering more questions accurately than GPT-4 on this benchmark is a testament to the strength of this approach. Again, this result must however be contextualised: Human performance on PubMedQA has been reported to be 78%, so what it means for a model to approach or even exceed this score remains unclear. Furthermore, as shown in the table, combining GPT-4 with MedPrompt as reported by Nori et al. (2023) allows the model to once again pull ahead and set the state-of-the-art on this benchmark. While BioLlama's score was achieved without any specialised MedPrompt technique, GPT-4's score was also recorded in the absence of retrieval augmentation. So perhaps it is hard to identify exactly what the difference in these scores implies.

On OpenQA scores, BioLlama trails far behind all other models. Notably, Frisoni et al. (2022) record a score of 43% on this benchmark, exceeding BioLlama's performance in spite of its massively larger parameter count. All proprietary models record much higher results. Perhaps, retrieval-augmenting on RCT200k was insufficient. Larger databases combined with further research on different training regimes could make BioLlama more competitive. However, BioLlama did improve upon Llama-2, and the proprietary models are more than 41x larger. As such, sheer parameter count may be the most important metric related to OpenQA performance.

Referring back to RQ6, I find that while BioLlama stays competitive and achieves some exciting results on QA, it struggles to keep up with larger and proprietary models on OpenQA. As with previous research questions, these results indicate that this approach deserves further exploration of the decisive factors in BioLlama's capabilities.

5 Discussion & Limitations

Having thoroughly explored BioLlama and how it behaves when tested on various downstream tasks, it is important to contextualise these findings into the wider picture. They do not exist in a vacuum: As elucidated in the introduction, the deployment of Llama-2-based models to healthcare contexts has to be treated as a realistic possibility. This investigation has set out to explore how a decoder-only model like Llama-2 can be RETRO-fitted, and whether this can make it more reliable.

As explored in Chapter 4, BioLlama generally exhibits higher scores than Llama-2. It seems to effectively retrieve from relevant papers, and successfully incorporates snippets from them to respond to questions with higher accuracy. Through this, it reaffirms the findings of Frisoni et al. (2022), who show that retrieval augmentation can improve the performance of LLMs on biomedical benchmarks. It goes further in demonstrating that RETRO-fitting can be done in the absence of an encoder. This is a substantial research finding, as most cutting-edge transformer models utilize decoder-only architectures. My results indicate that they too may potentially benefit from RETRO-fitting even though they lack encoders. A highlight of my findings in particular is BioLlama-13B's ability to outperform GPT-4 on PubMedQA. While this score comes with important caveats, it is indicative of the overall promises of my approach.

In addition to these strong points, it is crucial to also address the limitations of this work, many of which stem from its methodology. In particular, it is important to call into question whether the chosen benchmarks reliably track the properties of LLMs that could actually be of importance in their deployment to sensitive sectors. For example, while both of the employed QA tasks test the model's ability to incorporate a large amount of literature snippets when performing biomedical reasoning, it is unclear how this may translate to an indication of the model's behaviour when deployed in a real-world environment. Within more realistic contexts, it seems unlikely that there is much value in collating a number of PubMed excerpts and then prompting a model to use them for question answering. More likely might be that a system is tasked with collecting those excerpts on its own and then also using them to draw a conclusion (as originally intended in BioASQ). Nevertheless, it is doubtful whether a high performance on BioASQ5b as exemplified by both Llama-2 and BioLlama does translate into high reliability and safety in real, high-stakes scenarios. This same criticism can be applied to PubMedQA, which becomes harder to interpret as a result of the relatively "low" human performance of 78%.

A closer estimation of an LLM's biomedical knowledge may be provided by the chosen OpenQA tasks, as they do not include any contexts snippets. The model relies fully on its parameters, or in the case of BioLlama its retrieval database, to respond to queries. However, the degree to which this estimation is actually useful can also be criticised. For one, all used OpenQA benchmarks have multiple-choice structures, where again the correct answer is in a way included in the prompt (consider for example Figure 4.8). A harder task surely would be to ask the model questions with no choices to choose from at all. While this could not be marked as conveniently as in this investigation, it would provide a more relevant metric to inform decisions on real-world deployment. There could hypothetically be use in brainstorming 4-5 potential diagnoses and then gaining model feedback on which one is the most likely to be correct. However, it could be equally if not more important for the model to be able to suggest plausible diagnoses or treatments itself without already being provided with a choice. Finally, the way multiple-choice benchmarks

were approached in this investigation reveals a crucial methodology weakness. Models were prompted and fine-tuned to generate responses to questions on the very first token they produce. It has been shown in recent work (Wei et al. 2023) that LLMs become more reliable when they are asked to "think step-by-step", ie. provide a reasoning for responses. This makes them more interpretable, as they output observations and details that may have been relevant in reaching the conclusions they draw. In this work, BioLlama is not asked to produce any kind of reasoning at all. While inspecting the retrieved text for a question can make it more interpretable, it is tasked with immediately reaching a conclusion on the first token it predicts. This may be a crucial weakness and comprises an important research direction for future work. Likewise, the model is not given the option to indicate any form of uncertainty in its responses, or refuse to respond. It would be highly desirable for a system that is deployed to critical sectors to be able to indicate if it produces a prediction with a low or a high certainty, as this may inform the extent to which a user weights this output. So-called "refusals" (Anthropic 2024) are also a powerful tool for models like these. Through this option, they can potentially detect when a question they are prompted with is inappropriate or covers a topic they do not possess much knowledge over. BioLlama contains no such abilities beyond what Llama-2 features, and in fact in this approach is forced to produce an output for any and all questions. This too is a limitation that should be addressed in following explorations.

Similarly, in addition to issues with the employed prompting and benchmarking strategies, the way generated outputs are marked reveals important drawbacks too. As thoroughly discussed in Section 3.1, using Exact Match scores and LLM-as-a-judge introduces a number of limitations and biases to my approach. With regards to my use of the chosen benchmarks, prompting approaches and marking methodology, I follow conventions of the field to make results more comparable. However, I seriously question the extent to which they really measure the kind of biomedical knowledge LLMs possess. For example, I observe that when the same question is reworded slightly, such as changing the phrasing or replacing words with synonyms, models sometimes switch their answers. To what extent LLMs really "know" factoids is thus called into question. Meng et al. (2023) observe that factual associations in LLMs may be highly dependent on the specific phrasing they occur in. This is highly problematic for a domain as sensitive as healthcare. If high benchmark performances indicate that a model consistently answers questions on a specific topic correctly, this could be used to argue for the safety of its use in this particular domain. After deployment, however, it is very probable that the model is faced with formulations that may differ considerably from the ones in its training data and benchmark questions, as users are often much less predictable than engineering efforts assume. If such an unseen formulation on the same topic and factoid can cause a different prediction or assertion, this is a severe flaw of the model that calls its safety into question.

To summarize, it can be said that this investigation has produced a substantial research contribution in that it has thoroughly explored the potential of a novel retrieval augmentation technique. In spite of several limiting factors, BioLlama achieves notable results that prove it deserves further exploration. At the same time, numerous limitations stemming from the used methodology severely limit the extent to which any strong conclusions can be drawn from the presented results.

6 Conclusion

6.1 Summary and key takeaways

This dissertation introduces BioLlama, a decoder-only model that is retrieval-augmented on biomedical literature through an original RETRO-fit approach. In a first of its kind investigation, I demonstrate how to allow an LLM to perform chunked cross-attention in the absence of an encoder. To this end, I first experiment with few-shot learning, reproducing findings that it enables LLMs to reliably follow instructions. I then show that RETRO-fitting Llama-2 with my proposed method deteriorates benchmark performance if the added parameters remain randomly initialized. This is followed by results that elucidate how fine-tuning the new modules enables BioLlama to exceed Llama-2 baselines consistently. In order to gain a better understanding of how to configure this fine-tuning step, I additionally explore whether extending training to 2 epochs can be beneficial, finding no substantial impact. These findings are complemented by an in-depth comparison of BioLlama to Llama-2 on the MedMCQA benchmark specifically, revealing that BioLlama may outperform Llama-2 particularly on subjects that are very prominent in the corpus it retrieves text from. Finally, I contrast BioLlama's performance with other open-source and proprietary models, finding that BioLlama stays competitive on QA tasks and even outperforms GPT-4 by 2.3% on PubMedQA. However, it displays considerably weaker accuracy on OpenQA tasks, falling far behind proprietary models by Google Research (Singhal et al. 2022; 2023), as well as OpenAI and Microsoft Research (OpenAI et al. 2023; Nori et al. 2023).

6.2 Directions for future work

As a result of its comprehensive scope, this investigation incorporates a number of assumptions and design decisions. While they are at all times motivated by preliminary experimentation or findings from related works, this opens up a plethora of directions for future work. For example, the "multiple-choice" aspect of OpenQA questions could be removed to provide a harder benchmark that provides a better representation of the biomedical knowledge that the model possesses. Furthermore, the strategy used to mark model outputs could be changed from Exact Match or LLM-as-a-judge (Zheng et al. 2023) to manual marking, which would allow for the model to produce longer, Chain-of-Thought (Wei et al. 2023) outputs that provide better interpretability. In addition, as previously mentioned, incorporating not only neighbours but their continuations as in RETRO (Borgeaud et al. 2022) and BioReader (Frisoni et al. 2022) has the potential to substantially improve BioLlama's performance. As a whole, the retrieval component exposes a number of opportunities to try different approaches, such as a larger retrieval corpus (like the full PubMed database), comparing MedCPT with different retrievers, and investigating whether "relevance" as a metric to rank retrieved snippets should be replaced with something approximating "helpfulness". Similarly, the design choices in my RETRO-fit raises curiosities like the effect of RETRO-fitting more/other layers, varying the sizes of chunks, or on an ablation study of RETRO-fitting without an encoder as shown here or with a purpose built encoder to verify whether decoder-only RETRO-fitting does indeed make sense. Furthermore, recent research (Meng et al. 2023) suggests that factual associations are stored in feed-forward networks of LLMs, raising the question of whether they should be targeted instead of cross-attention modules in

retrieval augmentation. Finally, the chosen fine-tuning method could be supplemented with an initial pre-train of only the RETRO-fitted layers followed by a full fine-tune of all layers like in the work by Borgeaud et al. (2022) and Frisoni et al. (2022), as well as training only one model instead of a new one per benchmark.

To conclude on my work, I believe that it constitutes a substantial research contribution, but also features a number of limitations that restrict the kind of conclusions that can be drawn from it. While I show RETRO-fitting of decoder-only models to be both possible and lucrative, it also falls short of delivering the kind of performance gains it may have the potential to. This is the finding that I place the most weight on: While my retrieval augmentation has made Llama-2 more performant as measured by a number of biomedical benchmarks, it has far from eliminated hallucinations and fails to fully address explainability issues. With regards to my concerns surrounding the deployment of LLMs to sectors as sensitive as healthcare, I argue that these findings indicate that in spite of their advantages, the presented retrieval augmentation methods are insufficient to make LLMs as reliable or trustworthy as they would need to be to find their use in critical sectors. I emphasize that further research in this field is urgently necessary before introducing inconsistent black-boxes to hospitals, and hope that future work can help reduce these issues before they negatively affect vulnerable populations. In spite of the potential in improving world-wide access to high-quality healthcare, the possible threat to patient outcomes posed by opaque systems that are not yet fully understood needs to be taken seriously. I remain hopeful that the research groups working on the deployment of these systems to biomedicine carefully weigh up these risks and will reach decisions informed by rigorous and thorough experimentation.

A Appendices

A.1 Outline of source code

All source code is publicly inspectable at https://github.com/Neelectric/BioLlama. The readme.md and manual.md files provides further descriptions, but in essence, experiments were done by entering relevant details of a benchmark run in main.py. This driver file makes a call to inference.py, which handles the loading of the benchmark, promptifying, inference and saving results. Finally, main.py calls exact_match.py or judging.py, depending on whether Exact Match or LLM-as-a-judge is used, to mark results. These are then automatically written to the README, including a changelog. Note importantly that the files in the /src/ folder are all taken from the exllama library[1] and NOT written by me, all credits go to turboderp. I have also deleted the large benchmark and output files in my dissertation submission .zip file, though they are included in my GitHub/easily accessible online.

A.2 Hardware setup

The experiments reported in this dissertation were ran on a workstation that was generously supplied by Binaere Bauten GmbH. The following table includes the hardware specifications:

Table A.1: *Descriptions of the components of the workstation. Most important is the GPU setup and how much VRAM it comes with, as that limits the kind of LLMs that can be used. All other components are optimized to serve the GPUs with data as efficiently as possible.*

Component	Product	Relevant details
CPU	AMD-7950X3D	Up to 32 threads at 5.7 Ghz
GPUs	2x MSI GeForce RTX 4090 SUPRIM Liquid X	2x 24GB GPU VRAM
RAM	Corsair Vengeance 96GB DDR5	5600Mhz

Work was completed on the workstation with an SSH tunnel through OpenVPN. A VS Code server enabled persistent session management.

[1] https://github.com/turboderp/exllama

B BioLlama loss curves on QA tasks

On the QA tasks BioASQ5b and PubMedQA, BioLlama only has 486 and 1,000 questions available for training, respectively. As such, further information on these training runs is less informative than the OpenQA runs with 10,178/100,000 training samples, respectively. For the sake of completion, loss curves for the QA tasks are included below.

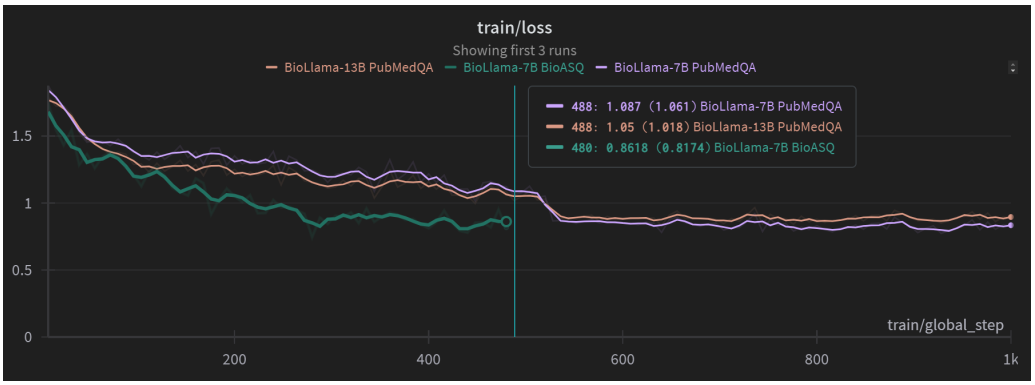


Figure B.1: *A graph showing the loss curves of BioLlama on PubMedQA and BioASQ. While the BioASQ loss seems to decrease further than the PubMedQA loss, this does not seem to translate into benchmark performance as BioLlama sees more gains through fine-tuning on PubMedQA.*

On these QA tasks, the training curves of BioLlama are roughly identical for 7B and 13B variants, and stagnate relatively early. This may be suggestive of the fact that the main adaptation for the new CCA parameters is to format the output as expected by the benchmark.

Bibliography

F. Ali. Gpt-1 to gpt-4: Each of openai's gpt models explained and compared, Apr 2023. URL https://www.makeuseof.com/gpt-models-explained-and-compared/.

R. Anil, S. Borgeaud, Y. Wu, J.-B. Alayrac, J. Yu, R. Soricut, J. Schalkwyk, A. M. Dai, A. Hauth, K. Millican, D. Silver, S. Petrov, M. Johnson, I. Antonoglou, J. Schrittwieser, A. Glaese, J. Chen, E. Pitler, T. Lillicrap, A. Lazaridou, O. Firat, J. Molloy, M. Isard, P. R. Barham, T. Hennigan, B. Lee, F. Viola, M. Reynolds, Y. Xu, R. Doherty, E. Collins, C. Meyer, E. Rutherford, E. Moreira, K. Ayoub, M. Goel, G. Tucker, E. Piqueras, M. Krikun, I. Barr, N. Savinov, I. Danihelka, B. Roelofs, A. White, A. Andreassen, T. von Glehn, L. Yagati, M. Kazemi, L. Gonzalez, M. Khalman, J. Sygnowski, A. Frechette, C. Smith, L. Culp, L. Proleev, Y. Luan, X. Chen, J. Lottes, N. Schucher, F. Lebron, A. Rrustemi, N. Clay, P. Crone, T. Kocisky, J. Zhao, B. Perz, D. Yu, H. Howard, A. Bloniarz, J. W. Rae, H. Lu, L. Sifre, M. Maggioni, F. Alcober, D. Garrette, M. Barnes, S. Thakoor, J. Austin, G. Barth-Maron, W. Wong, R. Joshi, R. Chaabouni, D. Fatiha, A. Ahuja, R. Liu, Y. Li, S. Cogan, J. Chen, C. Jia, C. Gu, Q. Zhang, J. Grimstad, A. J. Hartman, M. Chadwick, G. S. Tomar, X. Garcia, E. Senter, E. Taropa, T. S. Pillai, J. Devlin, M. Laskin, D. de Las Casas, D. Valter, C. Tao, L. Blanco, A. P. Badia, D. Reitter, M. Chen, J. Brennan, C. Rivera, S. Brin, S. Iqbal, G. Surita, J. Labanowski, A. Rao, S. Winkler, E. Parisotto, Y. Gu, K. Olszewska, Y. Zhang, R. Addanki, A. Miech, A. Louis, L. E. Shafey, D. Teplyashin, G. Brown, E. Catt, N. Attaluri, J. Balaguer, J. Xiang, P. Wang, Z. Ashwood, A. Briukhov, A. Webson, S. Ganapathy, S. Sanghavi, A. Kannan, M.-W. Chang, A. Stjerngren, J. Djolonga, Y. Sun, A. Bapna, M. Aitchison, P. Pejman, H. Michalewski, T. Yu, C. Wang, J. Love, J. Ahn, D. Bloxwich, K. Han, P. Humphreys, T. Sellam, J. Bradbury, V. Godbole, S. Samangooei, B. Damoc, A. Kaskasoli, S. M. R. Arnold, V. Vasudevan, S. Agrawal, J. Riesa, D. Lepikhin, R. Tanburn, S. Srinivasan, H. Lim, S. Hodkinson, P. Shyam, J. Ferret, S. Hand, A. Garg, T. L. Paine, J. Li, Y. Li, M. Giang, A. Neitz, Z. Abbas, S. York, M. Reid, E. Cole, A. Chowdhery, D. Das, D. Rogozińska, V. Nikolaev, P. Sprechmann, Z. Nado, L. Zilka, F. Prost, L. He, M. Monteiro, G. Mishra, C. Welty, J. Newlan, D. Jia, M. Allamanis, C. H. Hu, R. de Liedekerke, J. Gilmer, C. Saroufim, S. Rijhwani, S. Hou, D. Shrivastava, A. Baddepudi, A. Goldin, A. Ozturel, A. Cassirer, Y. Xu, D. Sohn, D. Sachan, R. K. Amplayo, C. Swanson, D. Petrova, S. Narayan, A. Guez, S. Brahma, J. Landon, M. Patel, R. Zhao, K. Villela, L. Wang, W. Jia, M. Rahtz, M. Giménez, L. Yeung, H. Lin, J. Keeling, P. Georgiev, D. Mincu, B. Wu, S. Haykal, R. Saputro, K. Vodrahalli, J. Qin, Z. Cankara, A. Sharma, N. Fernando, W. Hawkins, B. Neyshabur, S. Kim, A. Hutter, P. Agrawal, A. Castro-Ros, G. van den Driessche, T. Wang, F. Yang, S. yiin Chang, P. Komarek, R. McIlroy, M. Lučić, G. Zhang, W. Farhan, M. Sharman, P. Natsev, P. Michel, Y. Cheng, Y. Bansal, S. Qiao, K. Cao, S. Shakeri, C. Butterfield, J. Chung, P. K. Rubenstein, S. Agrawal, A. Mensch, K. Soparkar, K. Lenc, T. Chung, A. Pope, L. Maggiore, J. Kay, P. Jhakra, S. Wang, J. Maynez, M. Phuong, T. Tobin, A. Tacchetti, M. Trebacz, K. Robinson, Y. Katariya, S. Riedel, P. Bailey, K. Xiao, N. Ghelani, L. Aroyo, A. Slone, N. Houlsby, X. Xiong, Z. Yang, E. Gribovskaya, J. Adler, M. Wirth, L. Lee, M. Li, T. Kagohara, J. Pavagadhi, S. Bridgers, A. Bortsova, S. Ghemawat, Z. Ahmed, T. Liu, R. Powell, V. Bolina, M. Iinuma, P. Zablotskaia, J. Besley, D.-W. Chung, T. Dozat, R. Comanescu, X. Si, J. Greer, G. Su, M. Polacek, R. L. Kaufman, S. Tokumine, H. Hu, E. Buchatskaya, Y. Miao, M. Elhawaty, A. Siddhant, N. Tomasev, J. Xing, C. Greer, H. Miller, S. Ashraf, A. Roy, Z. Zhang, A. Ma, A. Filos, M. Besta, R. Blevins, T. Klimenko, C.-K. Yeh, S. Changpinyo, J. Mu, O. Chang, M. Pajarskas, C. Muir, V. Cohen, C. L. Lan,

K. Haridasan, A. Marathe, S. Hansen, S. Douglas, R. Samuel, M. Wang, S. Austin, C. Lan, J. Jiang, J. Chiu, J. A. Lorenzo, L. L. Sjösund, S. Cevey, Z. Gleicher, T. Avrahami, A. Boral, H. Srinivasan, V. Selo, R. May, K. Aisopos, L. Hussenot, L. B. Soares, K. Baumli, M. B. Chang, A. Recasens, B. Caine, A. Pritzel, F. Pavetic, F. Pardo, A. Gergely, J. Frye, V. Ramasesh, D. Horgan, K. Badola, N. Kassner, S. Roy, E. Dyer, V. Campos, A. Tomala, Y. Tang, D. E. Badawy, E. White, B. Mustafa, O. Lang, A. Jindal, S. Vikram, Z. Gong, S. Caelles, R. Hemsley, G. Thornton, F. Feng, W. Stokowiec, C. Zheng, P. Thacker, Çağlar Ünlü, Z. Zhang, M. Saleh, J. Svensson, M. Bileschi, P. Patil, A. Anand, R. Ring, K. Tsihlas, A. Vezer, M. Selvi, T. Shevlane, M. Rodriguez, T. Kwiatkowski, S. Daruki, K. Rong, A. Dafoe, N. FitzGerald, K. Gu-Lemberg, M. Khan, L. A. Hendricks, M. Pellat, V. Feinberg, J. Cobon-Kerr, T. Sainath, M. Rauh, S. H. Hashemi, R. Ives, Y. Hasson, Y. Li, E. Noland, Y. Cao, N. Byrd, L. Hou, Q. Wang, T. Sottiaux, M. Paganini, J.-B. Lespiau, A. Moufarek, S. Hassan, K. Shivakumar, J. van Amersfoort, A. Mandhane, P. Joshi, A. Goyal, M. Tung, A. Brock, H. Sheahan, V. Misra, C. Li, N. Rakićević, M. Dehghani, F. Liu, S. Mittal, J. Oh, S. Noury, E. Sezener, F. Huot, M. Lamm, N. D. Cao, C. Chen, G. Elsayed, E. Chi, M. Mahdieh, I. Tenney, N. Hua, I. Petrychenko, P. Kane, D. Scandinaro, R. Jain, J. Uesato, R. Datta, A. Sadovsky, O. Bunyan, D. Rabiej, S. Wu, J. Zhang, G. Vasudevan, E. Leurent, M. Alnahlawi, I. Georgescu, N. Wei, I. Zheng, B. Chan, P. G. Rabinovitch, P. Stanczyk, Y. Zhang, D. Steiner, S. Naskar, M. Azzam, M. Johnson, A. Paszke, C.-C. Chiu, J. S. Elias, A. Mohiuddin, F. Muhammad, J. Miao, A. Lee, N. Vieillard, S. Potluri, J. Park, E. Davoodi, J. Zhang, J. Stanway, D. Garmon, A. Karmarkar, Z. Dong, J. Lee, A. Kumar, L. Zhou, J. Evens, W. Isaac, Z. Chen, J. Jia, A. Levskaya, Z. Zhu, C. Gorgolewski, P. Grabowski, Y. Mao, A. Magni, K. Yao, J. Snaider, N. Casagrande, P. Suganthan, E. Palmer, G. Irving, E. Loper, M. Faruqui, I. Arkatkar, N. Chen, I. Shafran, M. Fink, A. Castaño, I. Giannoumis, W. Kim, M. Rybiński, A. Sreevatsa, J. Prendki, D. Soergel, A. Goedeckemeyer, W. Gierke, M. Jafari, M. Gaba, J. Wiesner, D. G. Wright, Y. Wei, H. Vashisht, Y. Kulizhskaya, J. Hoover, M. Le, L. Li, C. Iwuanyanwu, L. Liu, K. Ramirez, A. Khorlin, A. Cui, T. LIN, M. Georgiev, M. Wu, R. Aguilar, K. Pallo, A. Chakladar, A. Repina, X. Wu, T. van der Weide, P. Ponnapalli, C. Kaplan, J. Simsa, S. Li, O. Dousse, F. Yang, J. Piper, N. Ie, M. Lui, R. Pasumarthi, N. Lintz, A. Vijayakumar, L. N. Thiet, D. Andor, P. Valenzuela, C. Paduraru, D. Peng, K. Lee, S. Zhang, S. Greene, D. D. Nguyen, P. Kurylowicz, S. Velury, S. Krause, C. Hardin, L. Dixon, L. Janzer, K. Choo, Z. Feng, B. Zhang, A. Singhal, T. Latkar, M. Zhang, Q. Le, E. A. Abellan, D. Du, D. McKinnon, N. Antropova, T. Bolukbasi, O. Keller, D. Reid, D. Finchelstein, M. A. Raad, R. Crocker, P. Hawkins, R. Dadashi, C. Gaffney, S. Lall, K. Franko, E. Filonov, A. Bulanova, R. Leblond, V. Yadav, S. Chung, H. Askham, L. C. Cobo, K. Xu, F. Fischer, J. Xu, C. Sorokin, C. Alberti, C.-C. Lin, C. Evans, H. Zhou, A. Dimitriev, H. Forbes, D. Banarse, Z. Tung, J. Liu, M. Omernick, C. Bishop, C. Kumar, R. Sterneck, R. Foley, R. Jain, S. Mishra, J. Xia, T. Bos, G. Cideron, E. Amid, F. Piccinno, X. Wang, P. Banzal, P. Gurita, H. Noga, P. Shah, D. J. Mankowitz, A. Polozov, N. Kushman, V. Krakovna, S. Brown, M. Bateni, D. Duan, V. Firoiu, M. Thotakuri, T. Natan, A. Mohananey, M. Geist, S. Mudgal, S. Girgin, H. Li, J. Ye, O. Roval, R. Tojo, M. Kwong, J. Lee-Thorp, C. Yew, Q. Yuan, S. Bagri, D. Sinopalnikov, S. Ramos, J. Mellor, A. Sharma, A. Severyn, J. Lai, K. Wu, H.-T. Cheng, D. Miller, N. Sonnerat, D. Vnukov, R. Greig, J. Beattie, E. Caveness, L. Bai, J. Eisenschlos, A. Korchemniy, T. Tsai, M. Jasarevic, W. Kong, P. Dao, Z. Zheng, F. Liu, F. Yang, R. Zhu, M. Geller, T. H. Teh, J. Sanmiya, E. Gladchenko, N. Trdin, A. Sozanschi, D. Toyama, E. Rosen, S. Tavakkol, L. Xue, C. Elkind, O. Woodman, J. Carpenter, G. Papamakarios, R. Kemp, S. Kafle, T. Grunina, R. Sinha, A. Talbert, A. Goyal, D. Wu, D. Owusu-Afriyie, C. Du, C. Thornton, J. Pont-Tuset, P. Narayana, J. Li, S. Fatehi, J. Wieting, O. Ajmeri, B. Uria, T. Zhu, Y. Ko, L. Knight, A. Héliou, N. Niu, S. Gu, C. Pang, D. Tran, Y. Li, N. Levine, A. Stolovich, N. Kalb, R. Santamaria-Fernandez, S. Goenka, W. Yustalim, R. Strudel, A. Elqursh, B. Lakshminarayanan, C. Deck, S. Upadhyay, H. Lee, M. Dusenberry, Z. Li, X. Wang, K. Levin, R. Hoffmann, D. Holtmann-Rice, O. Bachem, S. Yue, S. Arora, E. Malmi, D. Mirylenka, Q. Tan, C. Koh, S. H. Yeganeh, S. Põder, S. Zheng, F. Pongetti, M. Tariq,

Y. Sun, L. Ionita, M. Seyedhosseini, P. Tafti, R. Kotikalapudi, Z. Liu, A. Gulati, J. Liu, X. Ye, B. Chrzaszcz, L. Wang, N. Sethi, T. Li, B. Brown, S. Singh, W. Fan, A. Parisi, J. Stanton, C. Kuang, V. Koverkathu, C. A. Choquette-Choo, Y. Li, T. Lu, A. Ittycheriah, P. Shroff, P. Sun, M. Varadarajan, S. Bahargam, R. Willoughby, D. Gaddy, I. Dasgupta, G. Desjardins, M. Cornero, B. Robenek, B. Mittal, B. Albrecht, A. Shenoy, F. Moiseev, H. Jacobsson, A. Ghaffarkhah, M. Rivière, A. Walton, C. Crepy, A. Parrish, Y. Liu, Z. Zhou, C. Farabet, C. Radebaugh, P. Srinivasan, C. van der Salm, A. Fidjeland, S. Scellato, E. Latorre-Chimoto, H. Klimczak-Plucińska, D. Bridson, D. de Cesare, T. Hudson, P. Mendolicchio, L. Walker, A. Morris, I. Penchev, M. Mauger, A. Guseynov, A. Reid, S. Odoom, L. Loher, V. Cotruta, M. Yenugula, D. Grewe, A. Petrushkina, T. Duerig, A. Sanchez, S. Yadlowsky, A. Shen, A. Globerson, A. Kurzrok, L. Webb, S. Dua, D. Li, P. Lahoti, S. Bhupatiraju, D. Hurt, H. Qureshi, A. Agarwal, T. Shani, M. Eyal, A. Khare, S. R. Belle, L. Wang, C. Tekur, M. S. Kale, J. Wei, R. Sang, B. Saeta, T. Liechty, Y. Sun, Y. Zhao, S. Lee, P. Nayak, D. Fritz, M. R. Vuyyuru, J. Aslanides, N. Vyas, M. Wicke, X. Ma, T. Bilal, E. Eltyshev, D. Balle, N. Martin, H. Cate, J. Manyika, K. Amiri, Y. Kim, X. Xiong, K. Kang, F. Luisier, N. Tripuraneni, D. Madras, M. Guo, A. Waters, O. Wang, J. Ainslie, J. Baldridge, H. Zhang, G. Pruthi, J. Bauer, F. Yang, R. Mansour, J. Gelman, Y. Xu, G. Polovets, J. Liu, H. Cai, W. Chen, X. Sheng, E. Xue, S. Ozair, A. Yu, C. Angermueller, X. Li, W. Wang, J. Wiesinger, E. Koukoumidis, Y. Tian, A. Iyer, M. Gurumurthy, M. Goldenson, P. Shah, M. Blake, H. Yu, A. Urbanowicz, J. Palomaki, C. Fernando, K. Brooks, K. Durden, H. Mehta, N. Momchev, E. Rahimtoroghi, M. Georgaki, A. Raul, S. Ruder, M. Redshaw, J. Lee, K. Jalan, D. Li, G. Perng, B. Hechtman, P. Schuh, M. Nasr, M. Chen, K. Milan, V. Mikulik, T. Strohman, J. Franco, T. Green, D. Hassabis, K. Kavukcuoglu, J. Dean, and O. Vinyals. Gemini: A family of highly capable multimodal models, 2023.

Anthropic. The claude 3 model family: Opus, sonnet, haiku, Mar 2024. URL `https://www-cdn.anthropic.com/de8ba9b01c9ab7cbabf5c33b80b7bbc618857627/Model_Card_Claude_3.pdf`.

N. Belrose, Z. Furman, L. Smith, D. Halawi, I. Ostrovsky, L. McKinney, S. Biderman, and J. Steinhardt. Eliciting latent predictions from transformers with the tuned lens, 2023.

S. Borgeaud, A. Mensch, J. Hoffmann, T. Cai, E. Rutherford, K. Millican, G. van den Driessche, J.-B. Lespiau, B. Damoc, A. Clark, D. de Las Casas, A. Guy, J. Menick, R. Ring, T. Hennigan, S. Huang, L. Maggiore, C. Jones, A. Cassirer, A. Brock, M. Paganini, G. Irving, O. Vinyals, S. Osindero, K. Simonyan, J. W. Rae, E. Elsen, and L. Sifre. Improving language models by retrieving from trillions of tokens, 2022.

T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei. Language models are few-shot learners, 2020.

N. D. Cao, W. Aziz, and I. Titov. Editing factual knowledge in language models, 2021.

D. Chen, A. Fisch, J. Weston, and A. Bordes. Reading wikipedia to answer open-domain questions, 2017.

Z. Chen, A. H. Cano, A. Romanou, A. Bonnet, K. Matoba, F. Salvi, M. Pagliardini, S. Fan, A. Köpf, A. Mohtashami, A. Sallinen, A. Sakhaeirad, V. Swamy, I. Krawczuk, D. Bayazit, A. Marmet, S. Montariol, M.-A. Hartley, M. Jaggi, and A. Bosselut. Meditron-70b: Scaling medical pretraining for large language models, 2023.

A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann, P. Schuh, K. Shi, S. Tsvyashchenko, J. Maynez, A. Rao,

P. Barnes, Y. Tay, N. Shazeer, V. Prabhakaran, E. Reif, N. Du, B. Hutchinson, R. Pope, J. Bradbury, J. Austin, M. Isard, G. Gur-Ari, P. Yin, T. Duke, A. Levskaya, S. Ghemawat, S. Dev, H. Michalewski, X. Garcia, V. Misra, K. Robinson, L. Fedus, D. Zhou, D. Ippolito, D. Luan, H. Lim, B. Zoph, A. Spiridonov, R. Sepassi, D. Dohan, S. Agrawal, M. Omernick, A. M. Dai, T. S. Pillai, M. Pellat, A. Lewkowycz, E. Moreira, R. Child, O. Polozov, K. Lee, Z. Zhou, X. Wang, B. Saeta, M. Diaz, O. Firat, M. Catasta, J. Wei, K. Meier-Hellstern, D. Eck, J. Dean, S. Petrov, and N. Fiedel. Palm: Scaling language modeling with pathways, 2022.

F. Dernoncourt and J. Y. Lee. Pubmed 200k rct: a dataset for sequential sentence classification in medical abstracts, 2017.

J. Dodge, M. Sap, A. Marasović, W. Agnew, G. Ilharco, D. Groeneveld, M. Mitchell, and M. Gardner. Documenting large webtext corpora: A case study on the colossal clean crawled corpus, 2021.

G. Frisoni, M. Mizutani, G. Moro, and L. Valgimigli. BioReader: a retrieval-enhanced text-to-text transformer for biomedical literature. In Y. Goldberg, Z. Kozareva, and Y. Zhang, editors, *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 5770–5793, Abu Dhabi, United Arab Emirates, Dec. 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.emnlp-main.390. URL https://aclanthology.org/2022.emnlp-main.390.

Y. Gu, R. Tinn, H. Cheng, M. Lucas, N. Usuyama, X. Liu, T. Naumann, J. Gao, and H. Poon. Domain-specific language model pretraining for biomedical natural language processing. *ACM Transactions on Computing for Healthcare*, 3(1):1–23, Oct. 2021. ISSN 2637-8051. doi: 10.1145/3458754. URL http://dx.doi.org/10.1145/3458754.

A. Gudibande, E. Wallace, C. Snell, X. Geng, H. Liu, P. Abbeel, S. Levine, and D. Song. The false promise of imitating proprietary llms, 2023.

R. Guo, P. Sun, E. Lindgren, Q. Geng, D. Simcha, F. Chern, and S. Kumar. Accelerating large-scale inference with anisotropic vector quantization. In *International Conference on Machine Learning*, 2020. URL https://arxiv.org/abs/1908.10396.

K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang. Realm: Retrieval-augmented language model pre-training, 2020.

G. Izacard and E. Grave. Leveraging passage retrieval with generative models for open domain question answering, 2021.

G. Izacard, M. Caron, L. Hosseini, S. Riedel, P. Bojanowski, A. Joulin, and E. Grave. Unsupervised dense information retrieval with contrastive learning, 2022a.

G. Izacard, P. Lewis, M. Lomeli, L. Hosseini, F. Petroni, T. Schick, J. Dwivedi-Yu, A. Joulin, S. Riedel, and E. Grave. Atlas: Few-shot learning with retrieval augmented language models, 2022b.

Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12): 1–38, Mar. 2023. ISSN 1557-7341. doi: 10.1145/3571730. URL http://dx.doi.org/10.1145/3571730.

D. Jin, E. Pan, N. Oufattole, W.-H. Weng, H. Fang, and P. Szolovits. What disease does this patient have? a large-scale open domain question answering dataset from medical exams, 2020.

Q. Jin, B. Dhingra, Z. Liu, W. W. Cohen, and X. Lu. Pubmedqa: A dataset for biomedical research question answering, 2019.

Q. Jin, Z. Yuan, G. Xiong, Q. Yu, H. Ying, C. Tan, M. Chen, S. Huang, X. Liu, and S. Yu. Biomedical question answering: A survey of approaches and challenges. *ACM Computing Surveys*, 55(2):1–36, Jan. 2022. ISSN 1557-7341. doi: 10.1145/3490238. URL http://dx.doi.org/10.1145/3490238.

Q. Jin, W. Kim, Q. Chen, D. C. Comeau, L. Yeganova, W. J. Wilbur, and Z. Lu. Medcpt: Contrastive pre-trained transformers with large-scale pubmed search logs for zero-shot biomedical information retrieval. *Bioinformatics*, 39(11), Nov. 2023. ISSN 1367-4811. doi: 10.1093/bioinformatics/btad651. URL http://dx.doi.org/10.1093/bioinformatics/btad651.

J. Johnson, M. Douze, and H. Jégou. Billion-scale similarity search with gpus, 2017.

V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. tau Yih. Dense passage retrieval for open-domain question answering, 2020.

J. Kasai, K. Sakaguchi, Y. Takahashi, R. L. Bras, A. Asai, X. Yu, D. Radev, N. A. Smith, Y. Choi, and K. Inui. Realtime qa: What's the answer right now?, 2022.

S. Kweon, J. Kim, J. Kim, S. Im, E. Cho, S. Bae, J. Oh, G. Lee, J. H. Moon, S. C. You, S. Baek, C. H. Han, Y. B. Jung, Y. Jo, and E. Choi. Publicly shareable clinical large language model built on synthetic clinical notes, 2023.

Y. Labrak, A. Bazoge, E. Morin, P.-A. Gourraud, M. Rouvier, and R. Dufour. Biomistral: A collection of open-source pretrained large language models for medical domains, 2024.

J. Lee, W. Yoon, S. Kim, D. Kim, S. Kim, C. H. So, and J. Kang. Biobert: a pre-trained biomedical language representation model for biomedical text mining. *Bioinformatics*, 36 (4):1234–1240, Sept. 2019a. ISSN 1367-4811. doi: 10.1093/bioinformatics/btz682. URL http://dx.doi.org/10.1093/bioinformatics/btz682.

K. Lee, M.-W. Chang, and K. Toutanova. Latent retrieval for weakly supervised open domain question answering, 2019b.

M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer. Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension, 2019.

P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks, 2021.

P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Comput. Surv.*, 55(9), jan 2023. ISSN 0360-0300. doi: 10.1145/3560815. URL https://doi.org/10.1145/3560815.

V. Liévin, C. E. Hother, A. G. Motzfeldt, and O. Winther. Can large language models reason about medical questions?, 2023.

Mayo-Clinic, Feb 2024. URL https://www.mayoclinic.org/drugs-supplements/lisinopril-oral-route/side-effects/drg-20069129?p=1.

K. Meng, D. Bau, A. Andonian, and Y. Belinkov. Locating and editing factual associations in gpt, 2023.

NHS, Dec 2021. URL https://www.nhs.uk/medicines/lisinopril/side-effects-of-lisinopril/.

H. Nori, Y. T. Lee, S. Zhang, D. Carignan, R. Edgar, N. Fusi, N. King, J. Larson, Y. Li, W. Liu, R. Luo, S. M. McKinney, R. O. Ness, H. Poon, T. Qin, N. Usuyama, C. White, and E. Horvitz. Can generalist foundation models outcompete special-purpose tuning? case study in medicine, 2023.

C. Olah and A. Jermyn. Reflections on qualitative research, Mar 2024. URL `https://transformer-circuits.pub/2024/qualitative-essay/index.html`.

OpenAI, :, J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat, R. Avila, I. Babuschkin, S. Balaji, V. Balcom, P. Baltescu, H. Bao, M. Bavarian, J. Belgum, I. Bello, J. Berdine, G. Bernadett-Shapiro, C. Berner, L. Bogdonoff, O. Boiko, M. Boyd, A.-L. Brakman, G. Brockman, T. Brooks, M. Brundage, K. Button, T. Cai, R. Campbell, A. Cann, B. Carey, C. Carlson, R. Carmichael, B. Chan, C. Chang, F. Chantzis, D. Chen, S. Chen, R. Chen, J. Chen, M. Chen, B. Chess, C. Cho, C. Chu, H. W. Chung, D. Cummings, J. Currier, Y. Dai, C. Decareaux, T. Degry, N. Deutsch, D. Deville, A. Dhar, D. Dohan, S. Dowling, S. Dunning, A. Ecoffet, A. Eleti, T. Eloundou, D. Farhi, L. Fedus, N. Felix, S. P. Fishman, J. Forte, I. Fulford, L. Gao, E. Georges, C. Gibson, V. Goel, T. Gogineni, G. Goh, R. Gontijo-Lopes, J. Gordon, M. Grafstein, S. Gray, R. Greene, J. Gross, S. S. Gu, Y. Guo, C. Hallacy, J. Han, J. Harris, Y. He, M. Heaton, J. Heidecke, C. Hesse, A. Hickey, W. Hickey, P. Hoeschele, B. Houghton, K. Hsu, S. Hu, X. Hu, J. Huizinga, S. Jain, S. Jain, J. Jang, A. Jiang, R. Jiang, H. Jin, D. Jin, S. Jomoto, B. Jonn, H. Jun, T. Kaftan, Łukasz Kaiser, A. Kamali, I. Kanitscheider, N. S. Keskar, T. Khan, L. Kilpatrick, J. W. Kim, C. Kim, Y. Kim, H. Kirchner, J. Kiros, M. Knight, D. Kokotajlo, Łukasz Kondraciuk, A. Kondrich, A. Konstantinidis, K. Kosic, G. Krueger, V. Kuo, M. Lampe, I. Lan, T. Lee, J. Leike, J. Leung, D. Levy, C. M. Li, R. Lim, M. Lin, S. Lin, M. Litwin, T. Lopez, R. Lowe, P. Lue, A. Makanju, K. Malfacini, S. Manning, T. Markov, Y. Markovski, B. Martin, K. Mayer, A. Mayne, B. McGrew, S. M. McKinney, C. McLeavey, P. McMillan, J. McNeil, D. Medina, A. Mehta, J. Menick, L. Metz, A. Mishchenko, P. Mishkin, V. Monaco, E. Morikawa, D. Mossing, T. Mu, M. Murati, O. Murk, D. Mély, A. Nair, R. Nakano, R. Nayak, A. Neelakantan, R. Ngo, H. Noh, L. Ouyang, C. O'Keefe, J. Pachocki, A. Paino, J. Palermo, A. Pantuliano, G. Parascandolo, J. Parish, E. Parparita, A. Passos, M. Pavlov, A. Peng, A. Perelman, F. de Avila Belbute Peres, M. Petrov, H. P. de Oliveira Pinto, Michael, Pokorny, M. Pokrass, V. Pong, T. Powell, A. Power, B. Power, E. Proehl, R. Puri, A. Radford, J. Rae, A. Ramesh, C. Raymond, F. Real, K. Rimbach, C. Ross, B. Rotsted, H. Roussez, N. Ryder, M. Saltarelli, T. Sanders, S. Santurkar, G. Sastry, H. Schmidt, D. Schnurr, J. Schulman, D. Selsam, K. Sheppard, T. Sherbakov, J. Shieh, S. Shoker, P. Shyam, S. Sidor, E. Sigler, M. Simens, J. Sitkin, K. Slama, I. Sohl, B. Sokolowsky, Y. Song, N. Staudacher, F. P. Such, N. Summers, I. Sutskever, J. Tang, N. Tezak, M. Thompson, P. Tillet, A. Tootoonchian, E. Tseng, P. Tuggle, N. Turley, J. Tworek, J. F. C. Uribe, A. Vallone, A. Vijayvergiya, C. Voss, C. Wainwright, J. J. Wang, A. Wang, B. Wang, J. Ward, J. Wei, C. Weinmann, A. Welihinda, P. Welinder, J. Weng, L. Weng, M. Wiethoff, D. Willner, C. Winter, S. Wolrich, H. Wong, L. Workman, S. Wu, J. Wu, M. Wu, K. Xiao, T. Xu, S. Yoo, K. Yu, Q. Yuan, W. Zaremba, R. Zellers, C. Zhang, M. Zhang, S. Zhao, T. Zheng, J. Zhuang, W. Zhuk, and B. Zoph. Gpt-4 technical report, 2023.

L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe. Training language models to follow instructions with human feedback, 2022.

A. Pal and M. Sankarasubbu. Gemini goes to med school: Exploring the capabilities of multimodal large language models on medical challenge problems & hallucinations, 2024.

A. Pal, L. K. Umapathi, and M. Sankarasubbu. Medmcqa : A large-scale multi-subject multi-choice dataset for medical domain question answering, 2022.

A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala. Pytorch: An imperative style, high-performance deep learning library, 2019.

L. N. Phan, J. T. Anibal, H. Tran, S. Chanana, E. Bahadroglu, A. Peltekian, and G. Altan-Bonnet. Scifive: a text-to-text transformer model for biomedical literature, 2021.

A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever. Improving language understanding by generative pre-training, 2018. URL https://openai.com/blog/improving-language-understanding-by-generative-pre-training/. Accessed: February 18, 2024.

P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang. SQuAD: 100,000+ questions for machine comprehension of text. In J. Su, K. Duh, and X. Carreras, editors, *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392, Austin, Texas, Nov. 2016. Association for Computational Linguistics. doi: 10.18653/v1/D16-1264. URL https://aclanthology.org/D16-1264.

N. Rimsky. Decoding intermediate activations in llama-2-7b, Jul 2023. URL https://www.lesswrong.com/posts/fJE6tscjGRPnK8C2C/decoding-intermediate-activations-in-llama-2-7b.

K. Singhal, S. Azizi, T. Tu, S. S. Mahdavi, J. Wei, H. W. Chung, N. Scales, A. Tanwani, H. Cole-Lewis, S. Pfohl, P. Payne, M. Seneviratne, P. Gamble, C. Kelly, N. Scharli, A. Chowdhery, P. Mansfield, B. A. y Arcas, D. Webster, G. S. Corrado, Y. Matias, K. Chou, J. Gottweis, N. Tomasev, Y. Liu, A. Rajkomar, J. Barral, C. Semturs, A. Karthikesalingam, and V. Natarajan. Large language models encode clinical knowledge, 2022.

K. Singhal, T. Tu, J. Gottweis, R. Sayres, E. Wulczyn, L. Hou, K. Clark, S. Pfohl, H. Cole-Lewis, D. Neal, M. Schaekermann, A. Wang, M. Amin, S. Lachgar, P. Mansfield, S. Prakash, B. Green, E. Dominowska, B. A. y Arcas, N. Tomasev, Y. Liu, R. Wong, C. Semturs, S. S. Mahdavi, J. Barral, D. Webster, G. S. Corrado, Y. Matias, S. Azizi, A. Karthikesalingam, and V. Natarajan. Towards expert-level medical question answering with large language models, 2023.

E. H. Soice, R. Rocha, K. Cordova, M. Specter, and K. M. Esvelt. Can large language models democratize access to dual-use biotechnology?, 2023.

R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang, and T. B. Hashimoto. Overview, 2023. URL https://crfm.stanford.edu/2023/03/13/alpaca.html.

S. Tian, Q. Jin, L. Yeganova, P.-T. Lai, Q. Zhu, X. Chen, Y. Yang, Q. Chen, W. Kim, D. C. Comeau, R. Islamaj, A. Kapoor, X. Gao, and Z. Lu. Opportunities and challenges for chatgpt and large language models in biomedicine and health. *Briefings in Bioinformatics*, 25(1), Nov. 2023. ISSN 1477-4054. doi: 10.1093/bib/bbad493. URL http://dx.doi.org/10.1093/bib/bbad493.

H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample. Llama: Open and efficient foundation language models, 2023a.

H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. S. Koura,

M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023b.

G. Tsatsaronis, G. Balikas, P. Malakasiotis, I. Partalas, M. Zschunke, M. R. Alvers, D. Weissenborn, A. Krithara, S. Petridis, D. Polychronopoulos, and et al. An overview of the bioasq large-scale biomedical semantic indexing and question answering competition. *BMC Bioinformatics*, 16(1), Apr 2015. doi: 10.1186/s12859-015-0564-6.

A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. *CoRR*, abs/1706.03762, 2017. URL `http://arxiv.org/abs/1706.03762`.

L. von Werra, Y. Belkada, L. Tunstall, E. Beeching, T. Thrush, N. Lambert, and S. Huang. Trl: Transformer reinforcement learning. `https://github.com/huggingface/trl`, 2020.

J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2023.

T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. L. Scao, S. Gugger, M. Drame, Q. Lhoest, and A. M. Rush. Huggingface's transformers: State-of-the-art natural language processing, 2020.

C. Wu, W. Lin, X. Zhang, Y. Zhang, Y. Wang, and W. Xie. Pmc-llama: Towards building open-source language models for medicine, 2023.

J. Wu, L. Ouyang, D. M. Ziegler, N. Stiennon, R. Lowe, J. Leike, and P. Christiano. Recursively summarizing books with human feedback, 2021.

L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena, 2023.

C. Zhou, P. Liu, P. Xu, S. Iyer, J. Sun, Y. Mao, X. Ma, A. Efrat, P. Yu, L. Yu, S. Zhang, G. Ghosh, M. Lewis, L. Zettlemoyer, and O. Levy. Lima: Less is more for alignment, 2023.